

Python Threading, Multiprocessing & concurrent.futures



УНИВЕРЗИТЕТ „ГОЦЕ ДЕЛЧЕВ“- ШТИП
ФАКУЛТЕТ ЗА ИНФОРМАТИКА
Компјутерско инженерство и технологии

СЕМИНАРСКА РАБОТА
ПАРАЛЕЛНО ПРОГРАМИРАЊЕ СО *Python*
Threading, Multiprocessing и concurrent.futures



Ментор,
Доц-Др. Душан Биков

Студент,
Мерсие Адемова
број на индекс:102856

Штип, декември 2025

Содржина

1. ВОВЕД.....	4
2. ТЕОРЕТСКА РАМКА ЗА ПАРАЛЕЛНО ПРОГРАМИРАЊЕ	4
2.1. GIL (GLOBAL INTERPRETER LOCK).....	5
2.2. ВИДОВИ ПАРАЛЕЛИЗАМ ВО PYTHON.....	6
2.2.1. <i>Thread – based</i> паралелизам.....	6
2.2.2. <i>Process – based</i> паралелизам.....	7
2.2.3. <i>Async</i> (асинхронно програмирање).....	7
3. Threading (НИШКИ) ВО PYTHON.....	8
3.1. <i>concurrent.futures</i> И СОВРЕМЕНОТО УПРАВУВАЊЕ СО НИШКИ	9
4. multiprocessing – ВИСТИНСКИ ПАРАЛЕЛИЗАМ ВО PYTHON	10
5. СПОРЕДБЕНА АНАЛИЗА: СЕРИСКО VS MULTIPROCESSING	12
5.1. ПРАКТИЧНА АНАЛИЗА СО ПРИМЕР	13
5.2. АНАЛИТИЧКА ДИСКУСИЈА	15
5.3. ЗАКЛУЧОК ОД АНАЛИЗАТА	16
6. ПРАКТИЧЕН ПРОЕКТ: СЕРИСКО СОРТИРАЊЕ НАСПРОТИ ПАРАЛЕЛНО СОРТИРАЊЕ – АНАЛИЗА СПОРЕД ГОЛЕМИНАТА НА ПОДАТОЧНИОТ СЕТ	16
6.1. ПРИСТАП НА ИМПЛЕМЕНТАЦИЈА.....	17
6.2. АЛГОРИТМИ ЗА СОРТИРАЊЕ.....	23
6.2.1. <i>Merge Sort</i>	23
6.2.2. <i>Quick Sort</i>	26
6.2.3. <i>Bubble Sort</i>	29
6.3. КОНЦЕПТ НА ДАЕМОН ПРОЦЕСИ И ИЗБЕГНУВАЊЕ ВО ПРОГРАМАТА	32
6.4. АНАЛИЗА НА МЕРЕЊЕТО И ПРЕТСТАВУВАЊЕТО НА РЕЗУЛТАТИТЕ ВО ПРОЕКТОТ	34
6.5. ОРГАНИЗАЦИЈА НА ЕКСПЕРИМЕНТОТ	36
6.6. АНАЛИЗА НА РЕЗУЛТАТИТЕ: ОДНЕСУВАЊЕ, ЗАБРЗУВАЊЕ И ЕФИКАСНОСТ СПОРЕД ГОЛЕМИНА НА ПОДАТОЧЕН СЕТ	38
7. ПРОШИРУВАЊЕ НА ЕКСПЕРИМЕНТАЛНАТА АНАЛИЗА НА ПАРАЛЕЛНОТО СОРТИРАЊЕ	52
7.1. КОМПАРАТИВНА АНАЛИЗА НА ИМПЛЕМЕНТАЦИЈАТА НА АЛГОРИТМИТЕ.....	54
7.2. АНАЛИЗА НА ЕКСПЕРИМЕНТАЛНИОТ И МЕРНИОТ МЕХАНИЗАМ	59
7.3. ГРАФИЧКИ КОРИСНИЧКИ ИНТЕРФЕЈС И ВИЗУЕЛИЗАЦИЈА	67
7.4. АВТОМАТСКИ РЕЖИМ НА ИЗВРШУВАЊЕ.....	71
7.5. ЦЕНТРАЛНА ЕКСПЕРИМЕНТАЛНА АНАЛИЗА	73
7.6. ИНТЕРПРЕТАЦИЈА НА РЕЗУЛТАТИТЕ.....	78
8. ЗАКЛУЧОК.....	87
9. КОРИСТЕНА ЛИТЕРАТУРА	88

1. Вовед

Современиот софтверски развој, особено во ерата на повеќе јадрените процесори, неизбежно вклучува концепти од паралелното програмирање. Поради зголемувањето на количината податоци, побарувачката за побрзо процесирање и потребата за одговорни апликации, програмерите се соочуваат со избор меѓу различни техники за паралелизам. Python, како еден од најкористените програмски јазици денес, нуди повеќе пристапи за конкурентно програмирање со цел да се подобри извршувањето на апликациите: моделот на нишки (*threading*), моделот на процеси (*multiprocessing*) и унифицираниот интерфејс преку модулот *concurrent.futures*.

Оваа семинарска работа ги опфаќа трите пристапи, нивната теорија, практична примена, предности и ограничувања. Особено внимание е посветено на тоа како Python ги реализира овие механизми, кои се разликуваат меѓу нив, и кои проблеми се јавуваат при конкурентно извршување на код.

2. Теоретска рамка за паралелно програмирање

Паралелното програмирање подразбира извршување на повеќе задачи истовремено. Сепак, важно е да се направи разлика меѓу вистински паралелизам и конкурентност. Конкурентноста се однесува на можноста програмот да се справува со повеќе задачи, но не нужно истовремено, туку со динамичка распределба на процесорското време. Вистинскиот паралелизам, пак, значи извршување на различни задачи навистина истовремено, што е возможно само на системи со повеќе процесорски јадра.

Паралелизмот претставува основен концепт во современото програмирање, чија цел е да се зголеми степенот на искористување на процесорските ресурси. Со развојот на хардверот и масовното усвојување на повеќе јадрените процесори, програмските јазици и платформи се обидуваат да понудат различни механизми

за конкурентно извршување. *Python* нуди повеќе модели за паралелизам, но со специфични ограничувања и архитектонски избори кои влијаат на тоа како и кога да се користат нишки, процеси или асинхронно програмирање.

2.1. GIL (Global Interpreter Lock)

Во *Python*, особено во најраспространетата имплементација *CPython*, паралелното извршување на нишки е ограничено од механизмот наречен **Global Interpreter Lock (GIL)**. *GIL* претставува глобална блокада која осигурува дека само една нишка може да извршува *Python* bytecode во даден момент. Иако повеќе нишки може да бидат активни, тие не може да извршуваат *Python* код истовремено – тие се менуваат во мали временски делови, но не работат паралелно на повеќе јадра.

Главната цел на *GIL* е да го поедностави управувањето со меморијата во *Python*, бидејќи *CPython* користи референтно броење (reference counting). Овој механизам би бил многу покомплициран без *GIL*, бидејќи би барал фина синхронизација за секој пристап до објекти и нивните референци.

Како резултат на ова:

- *Python* нишките **не обезбедуваат паралелен CPU – bound перформанс** – ако задачата е пресметковно интензивна, нишките ќе се блокираат меѓусебно.
- Затоа, за CPU – интензивни задачи како математика, обработка на слики, криптографија и машинско учење, нишките не го даваат очекуваното забрзување.
- *Multiprocessing* модулот го заобиколува *GIL* така што стартува повеќе процеси, секој со сопствен *Python* интерпретер и свој *GIL*.

Сепак, *GIL* **не е проблем за I/O – bound задачи**, бидејќи нишките најчесто чекаат влез/излез и не го држат *GIL* долго време. Во тие моменти, интерпретерот автоматски го предава *GIL* на друга нишка.

Благодарение на ова, нишките се практично решение за:

- Читање/пишување датотеки
- Работа со мрежни сокети
- HTTP барања
- Операции со бази на податоци
- GUI апликации

Со други зборови, GIL е ограничување само во специфичен тип задачи, но не и целосна пречка за паралелизам.

2.2. Видови паралелизам во Python

Python подржува три главни модели за конкурентно и паралелно извршување: *thread – based*, *process – based* и асинхроно програмирање. Секој од овие модели има различна примена, предности и слабости.

2.2.1. *Thread – based* паралелизам

Thread – based паралелизмот се реализира преку модулите `threading` и `concurrent.futures.ThreadPoolExecutor`. Овој модел подразбира работа со **нишки** кои го делат истиот мемориски простор.

Иако поради GIL нишките не може да извршуваат CPU операции паралелно, тие се исклучително ефикасни за I/O – bound задачи. Во вакви задачи, најголем дел од времето, апликацијата го поминува чекајќи – на одговор од сервер, на читање датотека или чекање на друг надворешен ресурс. За време на тоа чекање GIL се ослободува, дозволувајќи друга нишка да продолжи со извршување.

Thread-based моделот е најдобра опција кога имаме:

- Веб сервери со голем број истовремени барања
- Апликации со многу влезно/излезни операции
- Работа со многу датотеки истовремено
- Автоматизација која прави повеќе HTTP барања

2.2.2. *Process – based* паралелизам

Process – based паралелизмот е природно решение за задачи кои се фокусираат на CPU –интензивни операции. *Python* модулот *multiprocessing* создава независни процеси, секој со своја меморија и сопствен GIL. Тоа значи дека може да постои вистински паралелизам на сите јадра на процесорот.

Овој модел е идеален за:

- Научни пресметки
- Обработка на големи податочни множества
- Машинско учење
- Енкрипција
- Рендирање и обработка на слики

Процесите работат побавно при стартување и трошат повеќе меморија, но за CPU задачи тие се единственото правилно решение во *Python*.

2.2.3. *Async* (асинхроно програмирање)

Асинхроното програмирање претставува уште еден механизам за конкурентност во *Python*, имплементиран преку модулот *asyncio*. За разлика од нишките и процесите, асинхроните корутини **не сакаат да извршуваат код паралелно**, туку го прават програмот конкурентен преку кооперативно извршување.

Кога една корутина чека, таа експлицитно го враќа контролниот проток (преку *await*) и дозволува друга корутина да се извршува. Ова го прави *async* многу ефикасен систем за справување со илјадници мрежни операции без создавање голем број нишки.

3. *Threading* (нишки) во *Python*

Нишките претставуваат лесни единици на извршување кои го делат истиот мемориски простор во рамките на еден процес. Ова го прави нивното креирање многу побрзо во споредба со процесите. Програмската нишка функционира како

независна линија на извршување, но поради заедничката меморија, комуникацијата меѓу нишките е многу полесна и поефикасна.

Python обезбедува модел на нишки преку модулот **threading**, кој нуди класна и интуитивна реализација на овој концепт. Главната нишка, позната како *main thread*, е автоматски создадена при стартување на програмата. Од неа може да се стартуваат други нишки, кои паралелно ќе извршуваат различни делови од програмот.

Еден од најголемите предизвици при користење нишки во *Python* е појавата на т.н. *race conditions*. Овие ситуации настануваат кога две или повеќе нишки истовремено пристапуваат до ист ресурс – најчесто променлива во меморијата – и го менуваат резултатот непредвидливо. Пример за ваков проблем е кога повеќе нишки паралелно го инкрементираат истиот бројач без соодветна синхронизација. За да се избегнат вакви ситуации, се користат механизми за заклучување како *Lock*, кои обезбедуваат дека само една нишка може да пристапи до ресурсот во одреден момент.

Покрај *race conditions*, уште една потенцијална опасност е *deadlock*, состојба во која две нишки се блокираат меѓусебно, секоја чекајќи да се ослободи ресурс кој го држи другата. Поради овие опасности, работата со нишки бара внимателно планирање.

И покрај ограничувањата, нишките во *Python* се исклучително корисни кога задачата вклучува I/O операции – на пример читање и пишување датотеки, работа со мрежни конекции, барања кон веб сервери или работа со бази на податоци. Бидејќи овие операции претежно чекаат одговор од надворешни извори, *Python* лесно може да префрли извршување на друга нишка додека чека.

Токму затоа многу мрежни сервери, веб-краулери, апликации со повеќе интерфејси и графички апликации се базирани на нишки.


```

threadCreation.py > ...
1  import threading
2  import time
3
4  def worker():
5      print("Нитка стартува")
6      time.sleep(1)
7      print("Нитка заврши")
8
9  t = threading.Thread(target=worker)
10 t.start()
11 t.join()
12

```

1. Креирање на нишка со помош на модулот `threading`

3.1. *concurrent.futures* и современото управување со нишки

Со појавата на модулот *concurrent.futures*, Python доби модерен, унифициран API за управување и со нишки и со процеси. За работа со нишки се користи *ThreadPoolExecutor*, кој претставува менаџер на група предефинирани нишки.

Користењето *ThreadPoolExecutor* значително ја поедноставува паралелизацијата, бидејќи програмерот не мора рачно да креира, стартува и управува со животниот циклус на нишките. Доволно е да се предаде функција која треба да се изврши и Еxecutor автоматски ќе ја распредели задачата на следната слободна нишка. Ова го прави модулот исклучително погоден за апликации со голем број мали и независни I/O задачи.

```

ThreadPoolExecutor.py > ...
1  from concurrent.futures import ThreadPoolExecutor
2  import time
3
4  def task(n):
5      time.sleep(1)
6      return n * 2
7
8  with ThreadPoolExecutor(max_workers=5) as executor:
9      results = executor.map(task, [1, 2, 3, 4, 5])
10
11 print(list(results))
12

```

2. Употреба на модулот *concurrent.futures* во креирање на нишки

4. *multiprocessing* – вистински паралелизам во Python

Модулот **multiprocessing** претставува еден од најмоќните механизми во Python за постигнување вистински паралелизам. За разлика од нишките, кои го делат истиот мемориски простор и се ограничени од GIL, процесите функционираат како независни извршни единици. Секој процес има сопствен Python интерпретер и сопствен GIL, што овозможува паралелно извршување на кодот на повеќе јадрени современи процесори – нешто што не може да се постигне со обичните нишки во *CPython*.

Принципот на multiprocessing се заснова на тоа дека задачите се распределуваат меѓу повеќе процеси, при што секој процес работи одвоено и не го блокира извршувањето на останатите. Тоа овозможува значително подобрување на перформансите кај CPU – интензивните задачи, каде што computational load е голем, а секој пресметковен чекор може да работи паралелно со останатите. Примери за такви операции се алгоритмите за машинско учење, обработка на големи бројни низи, енкрипција и дешифрирање на податоци, обработка на фотографии и видеа, или симулации во природни и технички науки.

Наједноставниот начин да се започне со multiprocessing е користењето на класата **Process**, која претставува директна аналогија на *threading.Thread*, но со независност на процесот. При креирање нов процес, Python стартува целосно нов интерпретер и го извршува кодот во одделна мемориска околина. Ова е мошне корисно бидејќи го избегнува ограничувањето на GIL, но истовремено значи дека се “потешки” од нишките – трошат повеќе време за стартување и повеќе меморија.

Во практични сценарија, најчесто се користи **multiprocessing.Pool**, кој е дизајниран да менаџира група процеси. *Pool* овозможува ефикасно распределување на задачите: функциите се извршуваат паралелно врз повеќе аргументи или податочни групи, а модулот автоматски се грижи за синхронизација и враќање резултати. Тоа го прави *Pool* идеален за задачи кои имаат форма “функција + многу податоци”. Пример за ова е процесирање на голема листа броеви, каде секоја вредност може да се пресмета одделно.

Уште една значајна предност на multiprocessing е поддршката за различни типови комуникација меѓу процесите. Бидејќи секој процес има сопствена меморија, податоците се разменуваат преку механизми како **Queue**, **Pipe**, **Manager** или мемориски структури како **Value** и **Array**, кои создаваат контролирано споделување. Оваа архитектура ја прави комуникацијата побавна отколку кај нишките, но затоа со многу поголема стабилност и без опасност од класични проблеми како race conditions.

Модулот *concurrent.futures* дополнително ја поедноставува работата со процеси преку класата *ProcessPoolExecutor*, која ја апстрахира комплексноста на управување со процеси. Наместо рачно стартување процеси, распределба на задачи и собирање на резултати, програмерот може да користи интуитивни методи како *executor.submit()* или *executor.map()*, со што multiprocessing станува подостапен и практичен. Оваа апстракција им овозможува и на почетниците да искористат повеќе јадрен паралелизам без да влегуваат во сложени детали на архитектурата.

Потребно е да се истакне дека multiprocessing не е секогаш најдобрата опција. Започнувањето на многу процеси може да биде неефикасно поради големиот overhead, што значи дека е најсоодветен за задачи каде секоја поединечна операција е доволно тешка за да се оправда трошокот на нов процес. Тоа значи дека мали функции кои се извршуваат многу брзо нема да имаат значително забрзување; напротив, може дури да предизвикаат влошување на перформансите. Затоа, процесите најдобро функционираат кога задачите се долги и независни, а податоците се релативно стабилни или лесни за пренесување од еден процес во друг.

Исто така, multiprocessing внесува дополнителни предизвици во равојот, како потребата функциите да бидат *pickable* (серијализирачки), нешто што не важи за нишките. Python мора да може да ја “собере” функцијата и да ја испрати во нов процес, па затоа не може да се користат локални функции, *lambda* функцијата или функции дефинирани во интерактивна конзола.

И покрај овие ограничувања, *multiprocessing* останува еден од најважните механизми во Python за создавање високо ефикасни апликации кои ги користат сите расположливи ресурси на современите процесори. Неговата комбинација со *ProcessPoolExecutor* го прави моќен и истовремено едноставен за употреба, а во ситуации каде перформансите се критични, *multiprocessing* е единствениот начин да се постигне вистински паралелизам во Python.

5. Споредбена анализа: Сериско vs multiprocessing

За да се разбере придобивката од *multiprocessing* во Python, неопходно е да се изврши споредбена анализа меѓу сериско (секвенцијално) процесирање и паралелно процесирање со повеќе процесори. Оваа анализа ги опфаќа карактеристиките на двата пристапа, нивната примена, перформансите во реални услови и ограничувањата кои произлегуваат од архитектонските принципи на Python.

Сериското извршување претставува класичен модел во кој програмата ги обработува задачите една по една, секоја во свој ред. Овој пристап е едноставен, предвидлив и нема никакви проблеми со конкурентност, бидејќи во секој момент се извршува единствена линија на код. проблемот настанува кога имаме голем број задачи или задачи кои се пресметковно тешки. Во такви случаи, сериското извршување бргу станува неефикасно, особено на системи со повеќе јадрени процесори кои остануваат неискористени.

multiprocessing е дизајниран токму за овие ситуации. Наместо задачите да се извршуваат една по една, тие се распределуваат меѓу повеќе независни процеси. Секој процес има свој интерпретер и свој GIL, што овозможува вистинско паралелно извршување на повеќе CPU – интензивни операции истовремено. Со ова, времето на извршување може да се намали драматично, особено во ситуации каде операциите се независни една од друга и може да се распределат рамномерно меѓу процесите.

5.1. Практична анализа со пример

Следниот пример покажува сериско и паралелно пресметување на Фибоначи број за повеќе вредности. Ова е клсичен CPU – интензивен проблем кој соодветно ги илустрира разликите.

```
primeri > serialVSpallel.py > ...
1  import time
2  from concurrent.futures import ProcessPoolExecutor
3
4  def fib(n):
5      if n <= 1:
6          return n
7      return fib(n - 1) + fib(n - 2)
8
9  def run_serial():
10     numbers = [35, 35, 35, 35]
11     start = time.time()
12
13     results = [fib(n) for n in numbers]
14
15     end = time.time()
16     print("Сериско време:", end - start)
17     print("-----")
18
19  def run_parallel():
20     numbers = [35, 35, 35, 35]
21     start = time.time()
22
23     with ProcessPoolExecutor() as executor:
24         results = list(executor.map(fib, numbers))
25
26     end = time.time()
27     print("Паралелно време:", end - start)
28     print("-----")
29     print("Резултати:", results)
30
31  def main():
32     run_serial()
33     run_parallel()
34
35  if __name__ == "__main__":
36     main()
37
```

3. Сериско наспроти паралелно извршување на алгоритам за пресметување на Fibonacci низа со помош на модулот *concurrent.futures*

Во примерот, сите четири пресметки за сериското извршување се извршуваат една по една. За вредноста 35, пресметката трае релативно долго, па серија од четири вакви пресметки резултира со забележително вкупно време.

Додека, во вториот дел од кодот, во кој е претставено паралелно извршување, секој процес извршува една пресметка. На четири јадрен процесор, овие задачи навистина се извршуваат паралелно.

Резултатите од кодот се:

```
Сериско време: 6.615221261978149
-----
Паралелно време: 2.954465866088867
-----
Резултати: [9227465, 9227465, 9227465, 9227465]
```

4. Резултат од претходниот пример за пресметување на Фибоначи низа со помош на модулот *concurrent.futures*

Ова покажува дека multiprocessing овозможува приближно **3.5x до 4x брзина** кај CPU – интензивни задачи, што претставува речиси идеален speedup во согласност со бројот на јадра.

5.2. Аналитичка дискусија

Клучната причина за ова драматично забрзување е тоа што multiprocessing почнува независни процеси кои истовремено се извршуваат на повеќе јадра. Ова го прави целосно независен од GIL и со тоа идеален за операции кои бараат значителна пресметковна моќ.

Сериското извршување, и покрај тоа што е најлесно за имплементација, е ограничено од физичките ресурси на единечното извршување и не може да ги искористи повеќето јадра.

multiprocessing се покажува најкорисен кога:

- Задачите се долги и CPU – интензивни
- Задачите се независно една од друга
- Податоците можат да се поделат без сложена комуникација

Сепак, multiprocessing има и свои недостатоци. Секој процес користи одделен дел од меморијата, а пренесувањето податоци меѓу процесите има голем overhead од споделувањето на меморија кај нишки. За многу мали задачи, времето за стартување процеси може да ја надмине добивката од паралелното извршување.

5.3. Заклучок од анализата

Споредбената анализа покажува дека multiprocessing значително ги надминува перформансите на сериското извршување кај CPU – интензивните задачи. Скоро линеарниот speedup со бројот на процесорски јадра го прави multiprocessing незаменлив во домени како научни пресметки, машинско учење, обработка на слики и големи податочни системи.

Сепак, изборот меѓу сериско, *threading* и *multiprocessing* мора внимателно да се разгледува во зависност од природата на задачата. *Multiprocessing* е ефикасно решение за комплексни и тешки операции, но не секогаш оптимално за мали, брзи задачи или операции кои се I/O-bound.

Оваа анализа го потврдува фундаменталниот заклучок: **кога задачата е CPU – интензивна, multiprocessing е единствениот начин да се постигне вистински паралелизам во Python.**

6. Практичен проект: Сериско сортирање наспроти паралелно сортирање – Анализа според големината на податочниот сет

Сортирањето претставува идеална демонстрација за разликите меѓу сериското извршување, нишките и процесите, пред сè затоа што станува збор за CPU-bound задача. Во рамки на овој проект нема да се користи модулот *concurrent.futures*, иако тој нуди поедноставен и поапстрахиран интерфејс за работа со нишки и процеси. Причината за оваа одлука е методолошка: целта на анализата не е само практично да се постигне паралелизам, туку концептуално да се разберат механизмите што стојат зад него. Користењето на *threading* и *multiprocessing* директно овозможува појасна демонстрација на разликите во архитектурата,

управувањето со меморијата, комуникацијата меѓу извршувачките единици и влијанието на GIL.

Со ваков пристап, експерименталните резултати ќе ја одразуваат суштинската разлика меѓу конкурентност и вистински паралелизам, што претставува централна тема на ова истражување.

Проектот се состои од 3 алгоритми за сортирање:

- *Bubble Sort*, кој е бавен и одличен за демонстрација
- *Merge Sort*, рекурзивен и може добро да се паралелизира
- *Quick Sort*, брз алгоритам и покажува добри резултати во забрзувањето при големи податочни сетови.

За секој алгоритам ќе бидат направени мерења за сериското време, *threading* време и *process* време, при што ќе бидат генерирани табела со времиња и графички приказ на резултатите. Проектот е именуван како *serialSortingVSparallelSorting* и е составен од два модули:

- *sorting_algorithms.py* – во кој се сместени алгоритмите и
- *benchmark.py* – за мерење на различните времиња и визуелен приказ на резултатите.

Поврзувањето меѓу датотеките е изведено на начин кој ја запазува модуларноста: *benchmark.py* е директно поврзан со *sorting_algorithms.py*, кој пак е поврзан индиректно, преку *json* датотеката што ја генерира *benchmark.py*, што претставува типичен архитектонски шаблон за мали проекти.

6.1. Пристап на имплементација

Програмата е реализирана преку **паралелен програмски слој** над стандардниот програмски јазик Python. Овој слој овозможува извршување на делови од сортирачките алгоритми паралелно, без да се менува синтаксата или компајлерот на Python. Со помош на модулите *threading* и *multiprocessing*, програмата може да извршува ист алгоритам одделно на два или повеќе независни процеси или

нишки, овозможувајќи ја демонстрацијата на разликите меѓу сериско и паралелно сортирање.

Програмата ја следи Фостерова методологија за дизајн на паралелни алгоритми, која подразбира четири чекори:

- **Партиционирање** – Поделба на податоците и задачите на независни сегменти;
- **Комуникација** – пренос на резултати помеѓу паралелните единици.
- **Агломерација** – Комбинирање на резултатите во конечен излез;
- **Мапирање** – Применување на алгоритмот на секој сегмент;

Оваа структура ја покажува логичката организација на паралелното сортирање и објаснува зошто алгоритмите *Merge Sort* и *Quick Sort* се паралелизираат добро, додека *Bubble Sort* служи за демонстрација на концептот, иако е бавен за големи data-сетови.

Партиционирање претставува процес на делење на пресметките и податоците на делови. Доброто партиционирање ги разделува и пресметките и податоците на голем број многу мали делови. Можат да се разликуваат два типа на партиционирање:

- **Декомпозиција на доменот** – пристап при кој ги делиме податоците и одредуваме како да ги поврземе пресметките со податоците. Примери за овој пристап се делење на низите на поднизи, делење на јамките на подјамки, делење на простор на подпростори.
- **Функционална декомпозиција** – пристап при кој прво ги делиме функциите за пресметување на помали делови, а потоа одредуваме како да ги поврземе податоците со пресметките. На овој начин обично се добива колекција од задачи кои конкурентноста ја постигнуваат преку механизам на цевковод (pipeline).

Без разлика каква декомпозиција ќе избереме, секој новодобиен дел го нарекуваме **примитивна задача**. Колку има повеќе примитивни задачи, толку

подобро, бидејќи нивниот број претставува горна граница на паралелизмот што може да се постигне за дадениот проблем.

Критериумите за квалитетно партиционирање можат да се сумираат на следниот начин:

- Бројот на примитивни задачи треба да биде барем 10 пати поголем (еден ред големина) од бројот на процесори во целниот паралелен компјутер или кластер.
- Редундантните пресметки и редундантните структури на податоци се минимизирани.
- Примитивните задачи се приближно со иста големина, поради балансирањето меѓу процесорите.
- Бројот на задачи е растечка функција од големината на проблемот, поради посакуваната скалабилност.

Во конкретниот проект, листата што се сортира се дели на толкав број на делови колку што корисникот ќе зададе број на процеси и нишки соодветно, при што секој дел се обработува самостојно. Ова ја поставува основата за паралелно извршување и е особено корисно кај алгоритмите *Merge Sort* и *Quick Sort*, кои природно користат рекурзија и *divide & conquer* парадигма.

Комуникацијата е чекор во кој се одредуваат вредностите кои се пренесуваат меѓу формираните примитивни задачи. Постојат два основни вида комуникација:

- **Локална комуникација** – кога на задачата и се потребни вредности од мал број други примитивни задачи. Се креираат канали кои го опишуваат текот на тие податоци.
- **Глобална комуникација** – кога значаен број примитивни задачи мора да испратат податоци за да се изврши некоја пресметка. На пример, пресметување на збирот на сите вредности што ги чуваат примитивните задачи. Овие канали не се цртаат во раните фази на дизајнот.

Комуникацијата претставува дополнителен трошок кај паралелните алгоритми, бидејќи не постои кај секвинцијалните алгоритми. Минимизирањето на овој дополнителен товар е една од најважните цели на ефикасното паралелно програмирање.

Критериуми за добро дефинирана комуникација се:

- Комуникациските операции се балансираани меѓу задачите;
- Секоја задача комуницира само со мал број соседни задачи;
- Задачите можат истовремено да ги извршуваат комуникациите;
- Задачите можат истовремено да ги извршуваат пресметките.

Првите два чекори идентификуваат што е можно повеќе паралелизам. Последните два чекори ја земаат предвид целната архитектура.

Во проектот, нишките комуницираат преку заеднички листи (*shared memory*), додека процесите комуницираат преку *Pool.map*, кој пренесува делови на податоци во различни процеси и ги враќа сортираните резултати. Оваа комуникација овозможува да се комбинираат резултатите од паралелно извршените задачи.

Агломерација (Aggregation / Reduction) – ако бројот на задачи е за неколку реда големина поголем од бројот на процесори, самото нивно креирање претставува значителен дополнителен трошок. Агломерацијата е процес на групирање на задачите во поголеми задачи, со следниве цели:

- Подобрување на перформансите;
- Поедноставување на кодирањето;
- Најчесто целта е да се креира по една агломерирана задача по процесор, при што мапирањето станува тривијално;
- Намалување на комуникацискиот трошок преку зголемување на локалноста, односно целосно елиминирање на некоја комуникација со спојување на примитивните задачи кои комуницираат.

Друга можност е спојување на задачите – испраќачи од едната страна и задачите примачи од друга страна, со што се намалува бројот на испратени пораки. Поради карактеристиките на мрежниот хардвер, испраќањето една долга порака трае пократко од испраќањето повеќе кратки пораки со иста вкупна големина. Причината е латенцијата (доцнење на пораката), која претставува време потребно за започнување на испраќање на пораката и не зависи од нејзината должина.

При агломерацијата мора да се внимава да не се наруши скалабилноста на паралелниот дизајн, односно да не се ограничи можноста програмата да се извршува на систем со голем број процесори.

Критериуми за добра агломерација се:

- Локалноста на паралелниот алгоритам е зголемена;
- Повторените операции трошат помалку време од комуникациите што ги заменуваат;
- Репликацијата на податоците е доволно мала за да се овозможи скалирање;
- Агломерираниите задачи имаат слични трошоци за пресметка и комуникација;
- Бројот на задачи е растечка функција од големината на проблемот;
- Бројот на задачи е што е можно помал, но барем еднаков на бројот на процесори на целиот паралелен компјутер;
- Постои компромис помеѓу агломерацијата и количината на модификации во секвенцијалниот код.

Во проектот, агломерацијата се изведува преку функцијата *merge* кај *Merge Sort* и *Bubble Sort* и преку конкатенација кај *Quick Sort*. Агломерацијата ги обединува сортираните делови во целосно сортирана листа, зачувувајќи го правилниот редослед на елементите.

Мапирање е процес на доделување на агрегирани задачи на процесорите. Кај централизираните мултипроцесорски системи, мапирањето го извршува самиот оперативен систем. Кај системи со дистрибуирана меморија, мапирањето го извршува корисникот на паралелната програма.

Основните цели на мапирањето се максимизирање на искористеноста на процесорите и минимизирање на меѓупроцесната комуникација. Искористеноста на процесорите претставува просечен процент од времето во кое процесорите активно извршуваат задачи во исто време. Спротивно на тоа, неефикасно е кога еден процесор мирува додека другите работат. Меѓупроцесната комуникација се зголемува кога две задачи поврзани со канал се мапирани на различни процесори, а се намалува кога се мапирани на ист процесор.

Анализата покажува дека овие две цели се конфликтни. На пример ако имаме p процесори и сите задачи ги мапираме на еден процесор, комуникацијата е нула, но искористеноста на процесорите е $1/p$. Затоа, целта е да се најде добар баланс помеѓу максимизирањето на искористеноста и минимизирањето на комуникацијата. Сепак, наоѓањето на оптимално мапирање е NP-комплексен проблем, па затоа мора да се потпреме на хеуристики за да го решиме во конечно време.

Во програмата, секој дел се предава на соодветната сортирачка функција (*merge_sort_serial*, *quick_sort_serial* или *bubble_sort_serial*). Кога се користи *threading*, деловите се извршуваат во различни нишки, а кога се користи *multiprocessing* деловите се извршуваат во одделни процеси, овозможувајќи вистинска паралелизација на CPU-bound операции.

6.2. Алгоритми за сортирање

```
# =====  
# MERGE FUNCTION (за сите верзии)  
# =====  
def merge(left, right):  
    result = []  
    i = j = 0  
  
    while i < len(left) and j < len(right):  
        if left[i] <= right[j]:  
            result.append(left[i])  
            i += 1  
        else:  
            result.append(right[j])  
            j += 1  
  
    result.extend(left[i:])  
    result.extend(right[j:])  
    return result
```

5. Функцијата merge

6.2.1. Merge Sort

Сериска верзија

merge_sort_serial е рекурзивен алгоритам за сортирање кој користи divide & conquer стратегија. Листата се дели на две подлисти, кои се сортираат рекурзивно, а потоа се спојуваат со функцијата *merge*.

- Временска сложеност: $O(n \log n)$ во најдобар, просечен и најлош случај;
- Просторна сложеност: $O(n)$, поради потребата за нови листи при спојувањето;
- Придобивки: Стабилен алгоритам, добро предвидлив и лесно паралелизирачки делови.

```

# =====
# SERIAL MERGE SORT
# =====
def merge_sort_serial(arr):

    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort_serial(arr[:mid])
    right = merge_sort_serial(arr[mid:])

    return merge(left, right)

```

6. Функција *merge_sort_serial*

Threading верзија

Во *merge_sort_threading*, левата и десната половина се сортираат паралелно со две нишки. Секоја нишка ја користи сериската функција *merge_sort_serial*.

- Принцип: Партиционирање → паралелно мапирање → агрегација;
- Ограничување: Python GIL (Global Interpreter Lock) ја ограничува вистинската паралелизација на CPU- bound задачи, така што threading дава само минимално забрзување.

```

# =====
# THREADING MERGE SORT
# =====
def merge_sort_threading(arr):

    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left_part = arr[:mid]
    right_part = arr[mid:]

    left_result = []
    right_result = []

    def sort_left():
        left_result.append(merge_sort_serial(left_part))

    def sort_right():
        right_result.append(merge_sort_serial(right_part))

    t1 = threading.Thread(target=sort_left)
    t2 = threading.Thread(target=sort_right)

    t1.start()
    t2.start()

    t1.join()
    t2.join()

    return merge(left_result[0], right_result[0])

```

7. Функција *merge_sort_threading*

Multiprocessing верзија

Со *merge_sort_multiprocessing* секоја половина се извршува во независен процес преку *multiprocessing.Pool*. Ова овозможува вистинска паралелизација бидејќи секој процес има свој Python интерпретер и сопствен GIL.

- Принцип: Партиционирање → паралелно мапирање преку процеси → агрегација со *merge*.
- Предност: Вистинско забрзување кај големи data-сетови, особено CPU-bound.

```
# =====
# MULTIPROCESSING MERGE SORT
# =====
def merge_sort_multiprocessing(arr):

    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left_half = arr[:mid]
    right_half = arr[mid:]

    # Само едно ниво паралелизам → нема daemon проблем
    with multiprocessing.Pool(2) as pool:
        left, right = pool.map(
            merge_sort_serial,
            [left_half, right_half]
        )

    return merge(left, right)
```

8. Функција *merge_sort_multiprocessing*

6.2.2. Quick Sort

Сериска верзија

quick_sort_serial е рекурзивен алгоритам кој избира pivot елемент и ги дели елементите на три групи: помали од pivot, еднакви на pivot и поголеми од pivot. Потоа се сортираат помалите и поголемите делови рекурзивно.

- Временска сложеност: $O(n \log n)$ во просек, $O(n^2)$ во најлош случај (кога pivot е секогаш најмал или најголем елемент);
- Просторна сложеност: $O(\log n)$ рекурзивен стек;

- Придобивки: ефективен за големи data-сетови и добро се однесува во пракса, но не е стабилен алгоритам.

```
# =====  
# SERIAL QUICK SORT  
# =====  
def quick_sort_serial(arr):  
  
    if len(arr) <= 1:  
        return arr  
  
    pivot = arr[len(arr) // 2]  
  
    left = [x for x in arr if x < pivot]  
    mid = [x for x in arr if x == pivot]  
    right = [x for x in arr if x > pivot]  
  
    return (  
        quick_sort_serial(left)  
        + mid +  
        quick_sort_serial(right)  
    )
```

9. Функција *quick_sort_serial*

Threading верзија

Со и *quick_sort_threading* секоја половина од листата (помала и поголема од pivot) се сортира со сопствена нишка.

- Принцип: Партиционирање → паралелно мапирање со нишки → агрегација со конкатенација.
- Ограничување: GIL може да ја намали вистинската ефикасност за големи data-сетови.

```

# =====
# THREADING QUICK SORT
# =====
def quick_sort_threading(arr):

    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]

    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    left_result = []
    right_result = []

    def sort_left():
        left_result.append(quick_sort_serial(left))

    def sort_right():
        right_result.append(quick_sort_serial(right))

    t1 = threading.Thread(target=sort_left)
    t2 = threading.Thread(target=sort_right)

    t1.start()
    t2.start()

    t1.join()
    t2.join()

    return left_result[0] + mid + right_result[0]

```

10. Функција *quick_sort_threading*

Multiprocessing верзија

При *quick_sort_multiprocessing* мапирањето се извршува во независни процеси.

- Предност: Може да искористи повеќе CPU јадра, овозможувајќи вистинско паралелно сортирање.

```
# =====
# MULTIPROCESSING QUICK SORT
# =====
def quick_sort_multiprocessing(arr):

    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]

    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    with multiprocessing.Pool(2) as pool:
        left_sorted, right_sorted = pool.map(
            quick_sort_serial,
            [left, right]
        )

    return left_sorted + mid + right_sorted
```

11. Функција *quick_sort_multiprocessing*

6.2.3. Bubble Sort

Сериска верзија

bubble_sort_serial е класичен алгоритам со двојна јамка кој споредува соседни елементи и ги менува ако се во погрешен редослед.

- Временска сложеност: $O(n^2)$ во просек и најлош случај, $O(n)$ во најдобар случај (веќе сортирана листа);
- Просторна сложеност: $O(1)$ дополнителна меморија;
- Придобивки: Едноставен, лесен за разбирање и идеален за демонстрација на концептите на паралелно сортирање.

```

# =====
# SERIAL BUBBLE SORT
# =====
def bubble_sort_serial(arr):
    """
    Класичен сериски Bubble Sort
    """
    n = len(arr)
    arr = arr.copy() # Копија за да не се менува оригиналниот list
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j] # Swap
    return arr

```

12. Функција *bubble_sort_serial*

Threading верзија

Кај *bubble_sort_threading* листата се дели на две половини, секоја се сортира во своја нишка, а потоа резултатите се спојуваат со *merge*.

- Ограничување: Поради $O(n^2)$ сложеност, паралелизацијата не дава големо забрзување на големи data-сетови;
- Придобивка: Добра за академска демонстрација на threading концептите.

```

# =====
# THREADING BUBBLE SORT
# =====
def bubble_sort_threading(arr):
    """
    Bubble Sort со 2 нишки за демонстрација, делиме листа на две половини
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_part = arr[:mid]
    right_part = arr[mid:]

    left_result = []
    right_result = []

    def sort_left():
        left_result.append(bubble_sort_serial(left_part))

    def sort_right():
        right_result.append(bubble_sort_serial(right_part))

    t1 = threading.Thread(target=sort_left)
    t2 = threading.Thread(target=sort_right)
    t1.start()
    t2.start()
    t1.join()
    t2.join()

    # Спојување на сортираните половини
    return merge(left_result[0], right_result[0])

```

13. Функција *bubble_sort_threading*

Multiprocessing верзија

Кај *bubble_sort_multiprocessing* секоја половина се сортира паралелно со различни процеси. Резултатите се спојуваат со *merge*.

- Предност: Може да се види вистинско паралелно извршување, но сепак ограничено од бавноста на *Bubble Sort*.

```
# =====
# MULTIPROCESSING BUBBLE SORT
# =====
def bubble_sort_multiprocessing(arr):
    """
    Bubble Sort со multiprocessing [ ] делиме на два процеси
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = arr[:mid]
    right_half = arr[mid:]

    with multiprocessing.Pool(2) as pool:
        left_sorted, right_sorted = pool.map(
            bubble_sort_serial,
            [left_half, right_half]
        )

    return merge(left_sorted, right_sorted)
```

14. Функција *bubble_sort_multiprocessing*

6.3. Концепт на daemon процеси и избегнување во програмата

Во паралелното програмирање, секој процес или нишка може да биде daemon или non-daemon.

- Daemon процес/нишка е помошен процес кој работи во позадина за поддршка на главниот процес. Тој се завршува автоматски кога главниот процес ќе заврши, без разлика дали daemon процесот завршил со својата задача или не.
- Non-daemon процес/нишка се извршува независно и го блокира завршувањето на главниот процес додека не заврши целосно.

Проблемот со daemon процесите е што ако главниот процес заврши, сите daemon процеси се принудно прекинати, што може да резултира со непотполно сортирање или загуба на резултати. Ова е особено важно во *Python multiprocessing*, каде процесите по дифолт може да се третираат како daemon, а рекурзивни *multiprocessing* решенија може да предизвикаат *RuntimeError* поради обид за создавање на нови процеси од daemon процес.

Во оваа програма, овој проблем е свесно избегнат на два начини:

1. Едно ниво на паралелизам:

- Во *merge_sort_multiprocessing*, *quick_sort_multiprocessing* и *bubble_sort_multiprocessing* се користи само едно ниво на *multiprocessing Pool* (2 процеси);
- Рекурзивното повикување на сервиските сортирачки функции се извршува внатре во секој процес без да се создава нов *Pool*, што значи дека нема daemon процеси кои се обидуваат да создадат свои деца.

```
# Само едно ниво паралелизам → нема daemon проблем
with multiprocessing.Pool(2) as pool:
```

15. Прв начин за спречување на појава на daemon процеси

2. Сигурен entry point со *if __name__ == "__main__":* и *freeze_support()*:

- На Windows, *multiprocessing* бара секој код што создава процеси да биде заштитен со *if __name__ == "__main__":* за да се избегне неочекувано реизвршување на кодот при создавање на нови процеси;
- *multiprocessing.freeze_support()* овозможува програмата да биде Windows-safe и да не се појави проблем со daemon процеси кога се користи *multiprocessing Pool*.


```
# -----
# 5. ENTRY POINT (IMPORTANT FOR MULTIPROCESSING)
# -----
if __name__ == "__main__":
    multiprocessing.freeze_support()
    main()
```

16. Втор начин за спречување на појава на даемон процеси

Со ова сите паралелни сортирање ќе се извршуваат стабилно, сортираните резултати ќе бидат целосни и точни и нема да се појавуваат *RuntimeError* или непредвидливи прекинувања поради даемон процеси.

6.4. Анализа на мерењето и претставувањето на резултатите во проектот

Во рамките на овој проект, модулот *benchmark.py* ја има улогата на експериментална и аналитичка компонента, преку која се врши квантитативна евалуација на перформансите на имплементираниите сортирачки алгоритми. Неговата функција не е да сортира податоци, туку да го мери, споредува и визуелизира начинот на нивното извршување во различни паралелни модели.

Мерењето на извршното време е реализирано преку функцијата *benchmark(func, data)*, која претставува апстракција на експериментален тајмер. Оваа функција го енкапсулира процесот на следење на времетраењето на извршување на даден алгоритам, при што:

- Во почетокот се повикува *time.time()* за да се регистрира почетниот временски момент;
- Потоа се извршува сортирачка функција над копија од податочното множество;
- По завршувањето, повторно се повикува *time.time()* за да се регистрира крајниот момент;
- Разликата меѓу двата временски маркери го дава вкупното извршно време.

На овој начин, мерењето е изолирано од останатите активности во програмата, што обезбедува релативно објективна проценка на перформансите.

```
# -----  
# 2. BENCHMARK TIMER  
# -----  
def benchmark(func, data):  
    start = time.time()  
    func(data)  
    end = time.time()  
    return end - start
```

17. Функција за креирање на тајмер за мерење на брзината на извршување на различните пристапи

Особено е важно што се користи `data.copy()`, со што се обезбедува секој алгоритам да работи над идентично, несортирано множество. Ова ја зачувува експерименталната валидност, бидејќи сортирањето на веќе сортирана листа би дало искривени резултати.

```
for name, funcs in algorithms:  
  
    print(f"Running {name} Serial ...")  
    t_serial = benchmark(funcs[0], data.copy())  
    print(f"{name} Serial: {t_serial:.4f} sec\n")  
  
    print(f"Running {name} Threading ...")  
    t_thread = benchmark(funcs[1], data.copy())  
    print(f"{name} Threading: {t_thread:.4f} sec\n")  
  
    print(f"Running {name} Multiprocessing ...")  
    t_process = benchmark(funcs[2], data.copy())  
    print(f"{name} Multiprocessing: {t_process:.4f} sec\n")  
  
    results[name] = [t_serial, t_thread, t_process]
```

18. Копирање на истата несортирана низа која ќе биде доделена на алгоритмите за сортирање

Функцијата `generate_dataset(size)` генерира случајна листа од цели броеви во зададен опсег. Овој dataset има двојна улога:

- Претставува влезен проблем за алгоритмите;
- Овозможува контролирање на комплексноста преку параметарот *size*, кој претставува компромис меѓу доволна големина за да се почувствува паралелизација и практичното време на извршување, особено за Bubble Sort кој има квадратна сложеност.

```
# -----
# 1. DATASET
# -----
def generate_dataset(size):
    return [random.randint(1, 100000) for _ in range(size)]
```

19. Дефинирање на големината на сетот на податоци

6.5. Организација на експериментот

Експериментот е структуриран преку листата `algorithms`, која претставува експериментална матрица составена од:

- Име на алгоритам;
- Три извршни модели:
 - Сериски
 - Threading
 - Multiprocessing

Оваа структура овозможува унифицирано тестирање, каде секој алгоритам поминува низ истата експериментална процедура.

```

algorithms = [
    ("Merge Sort", [
        merge_sort_serial,
        merge_sort_threading,
        merge_sort_multiprocessing
    ]),
    ("Quick Sort", [
        quick_sort_serial,
        quick_sort_threading,
        quick_sort_multiprocessing
    ]),
    ("Bubble Sort", [
        bubble_sort_serial,
        bubble_sort_threading,
        bubble_sort_multiprocessing
    ]),
]

```

20. Низа од алгоритмите и различните пристапи на извршување на сортирањето според конкретен алгоритам

По извршувањето на мерењата, резултатите се запишуваат во JSON датотека *results.json*. Овој пристап овозможува подоцнежна анализа, споредба и визуелизација.

```

# Save JSON
with open("results.json", "w") as f:
    json.dump(results, f, indent=4)

print("Results saved → results.json")

```

21. Кирање на json датотека и запишување на резултатите во неа

Визуелната анализа е реализирана преку функцијата *visualize_results(results)*, која претставува графичка интерпретација на измерените перформанси.

```

# -----
# 3. VISUALIZATION
# -----
def visualize_results(results):

    algorithms = list(results.keys())

    serial_times = [results[a][0] for a in algorithms]
    thread_times = [results[a][1] for a in algorithms]
    process_times = [results[a][2] for a in algorithms]

    x = np.arange(len(algorithms))
    bar_width = 0.25

    plt.figure(figsize=(10, 6))

    plt.bar(x - bar_width, serial_times, width=bar_width, Label="Serial")
    plt.bar(x, thread_times, width=bar_width, Label="Threading")
    plt.bar(x + bar_width, process_times, width=bar_width, Label="Multiprocessing")

    plt.xticks(x, algorithms)
    plt.ylabel("Time (seconds)")
    plt.title("Parallel vs Serial Sorting Benchmark")
    plt.legend()

    plt.grid(axis="y", linestyle="--", alpha=0.6)

    plt.tight_layout()
    plt.show()

```

22. Визуелизација на резултатите

Податоците се трансформираат во три временски вектори: *serial_times*, *thread_times* и *process_times*. Овие вектори овозможуваат директна споредба на извршните модели. На x-оската се позиционирани алгоритмите, додека y-оската го претставува времето на извршување во секунди.

6.6. Анализа на резултатите: однесување, забрзување и ефикасност според големина на податочен сет

Практичниот експеримент за споредба на сериско и паралелно сортирање беше изведен врз повеќе податочни сетови со прогресивно зголемување на обемот:

5 000, 10 000, 15 000, 20 000, 25 000, 30 000, 50 000, 1 000 000, 2 000 000, 3 000 000 и 5 000 000 елементи.

Целта на ваквата градација беше да се следи не само апсолутното време на извршување, туку и еволуцијата на паралелното забрзување (*speedup*) и процесорската ефикасност (*efficiency*) како функции од пресметковната грануларност.

Кај најмалите податочни сетови доминира ефектот на извршувачкиот overhead.

Сериските верзии на сите алгоритми покажуваат најкратко време на извршување, бидејќи нема креирање на нишки или процеси, нема синхронизација и нема комуникациски трошок.

Bubble Sort овде е релативно брз поради малиот обем. *Threading* и *multiprocessing* не носат реално забрзување. *Speedup* » 1, а ефикасноста е многу ниска, бидејќи времето за креирање процеси е споредливо со времето на самото сортирање.

Merge Sort threading покажува минимални варијации, но без вистинско забрзување поради GIL. *Multiprocessing* е дури и побавен од серискиот, што резултира со негативно забрзување

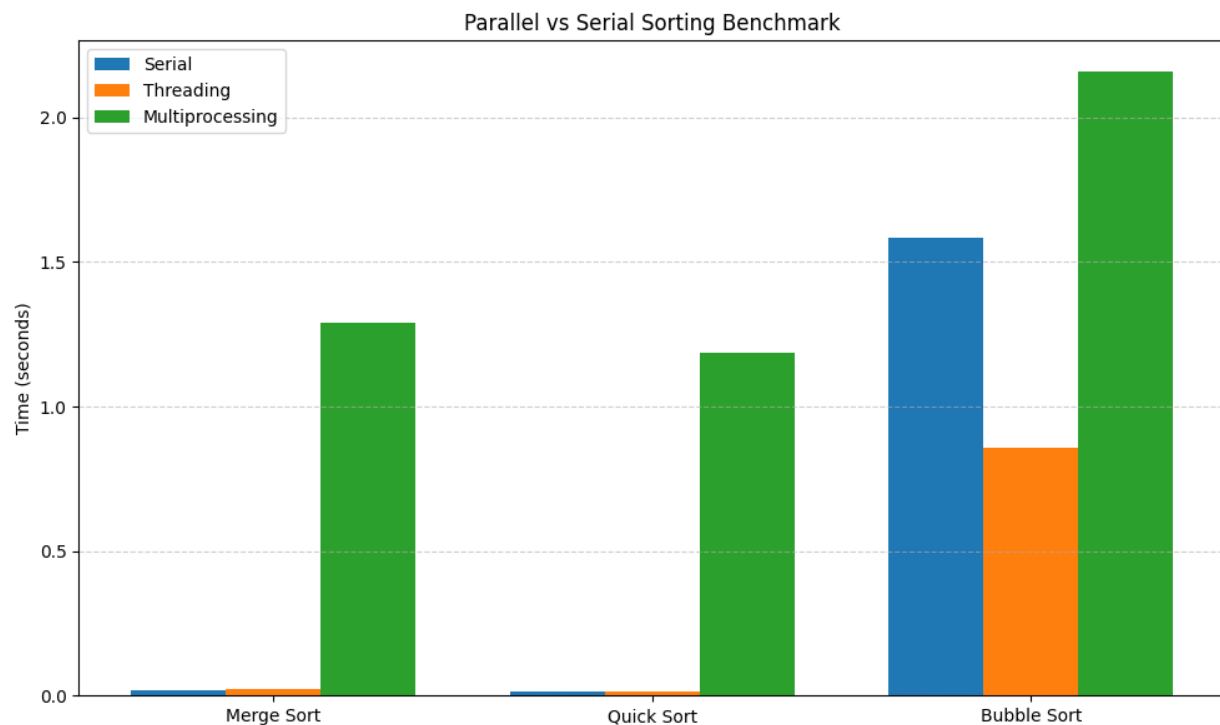
Quick Sort сериски е најбрз. Паралелните верзии страдаат од overhead. Ефикасноста е ниска бидејќи процесорите не добиваат доволно работа.

```

1  {
2      "Merge Sort": [
3          0.028191089630126953,
4          0.03844642639160156,
5          1.605565071105957
6      ],
7      "Quick Sort": [
8          0.012918472290039062,
9          0.014285564422607422,
10         1.0785157680511475
11     ],
12     "Bubble Sort": [
13         2.0573859214782715,
14         0.955782413482666,
15         1.8661620616912842
16     ]
17 }

```

23. Резултати при мерењето на сет од 5 000 податоци



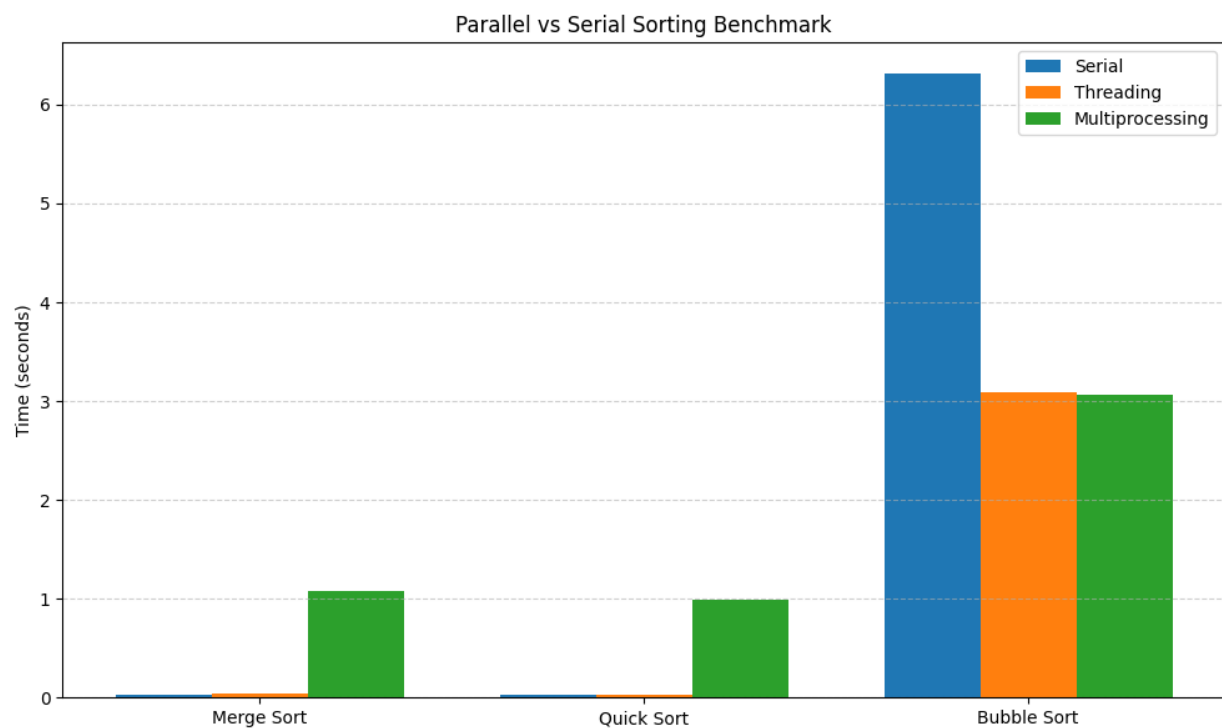
24. Визуелен приказ од резултатот при мерењето на сет од 5 000 податоци

```

1 {
2   "Merge Sort": [
3     0.031530141830444336,
4     0.03003215789794922,
5     0.9186561107635498
6   ],
7   "Quick Sort": [
8     0.022218704223632812,
9     0.023481369018554688,
10    0.8565433025360107
11  ],
12  "Bubble Sort": [
13    5.704411268234253,
14    2.752148866534424,
15    2.612393856048584
16  ]
17 }

```

25. Резултати при мерењето на сет од 10 000 податоци



26. Визуелен приказ од резултатот при мерењето на сет од 10 000 податоци

Со сетовите од 15 000 до 30 000 елементи започнува транзицијата од overhead-доминантно кон пресметковно-доминантно извршување.

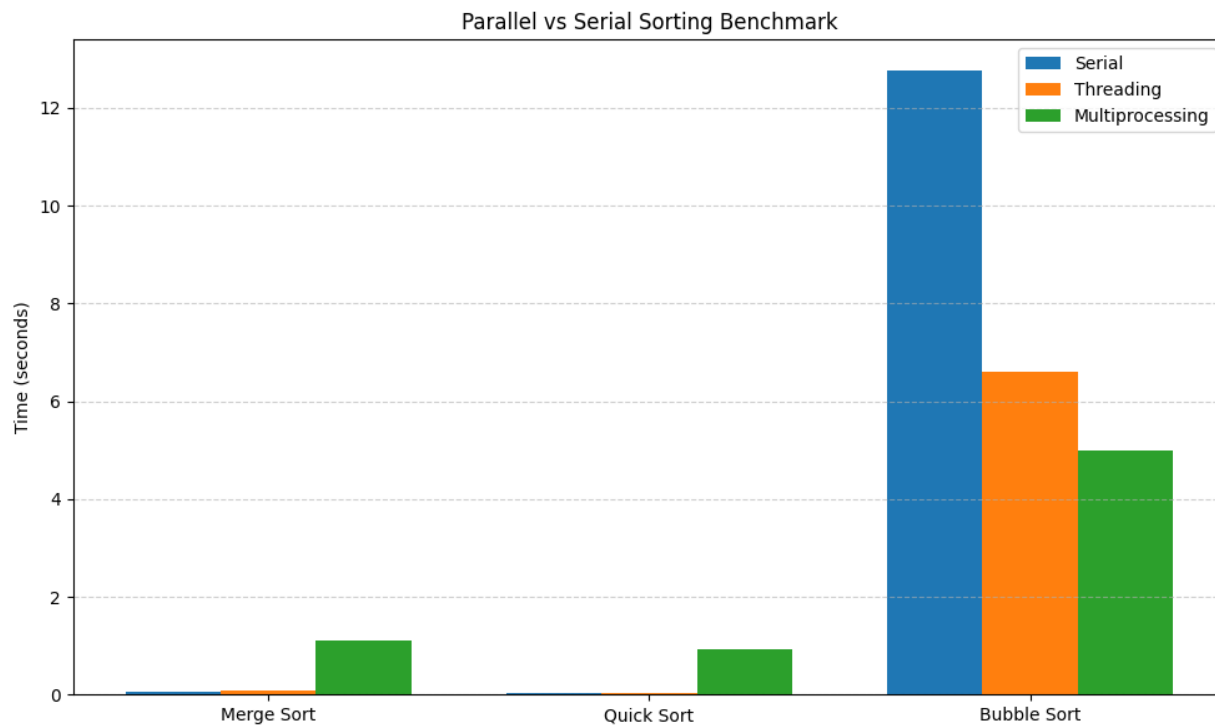
При **Bubble Sort** времето расте квадратично. *Threading* не помага, а *multiprocessing* дава мало подобрување, но забрзувањето останува ниско: $Speedup < 2$. Ефикасноста е ограничена поради зависностите меѓу итерациите.

Merge Sort multiprocessing почнува да покажува реално забрзување. Партиционирањето на две половини создава доволно работа за секој процес. Ефикасноста се подобрува бидејќи комуникацискиот трошок станува релативно помал.

Quick Sort multiprocessing покажува повисоко забрзување од *Merge Sort*, особено кога pivot создава балансирани партии. Ефикасноста е повисока, но варијабилна.

```
© results.json > ...
1  {
2    "Merge Sort": [
3      0.04245710372924805,
4      0.05630230903625488,
5      0.9339582920074463
6    ],
7    "Quick Sort": [
8      0.04115462303161621,
9      0.039639949798583984,
10     0.9255621433258057
11   ],
12   "Bubble Sort": [
13     13.149041175842285,
14     6.927908897399902,
15     5.084687948226929
16   ]
17 }
```

27. Резултати при мерењето на сет од 15 000 податоци



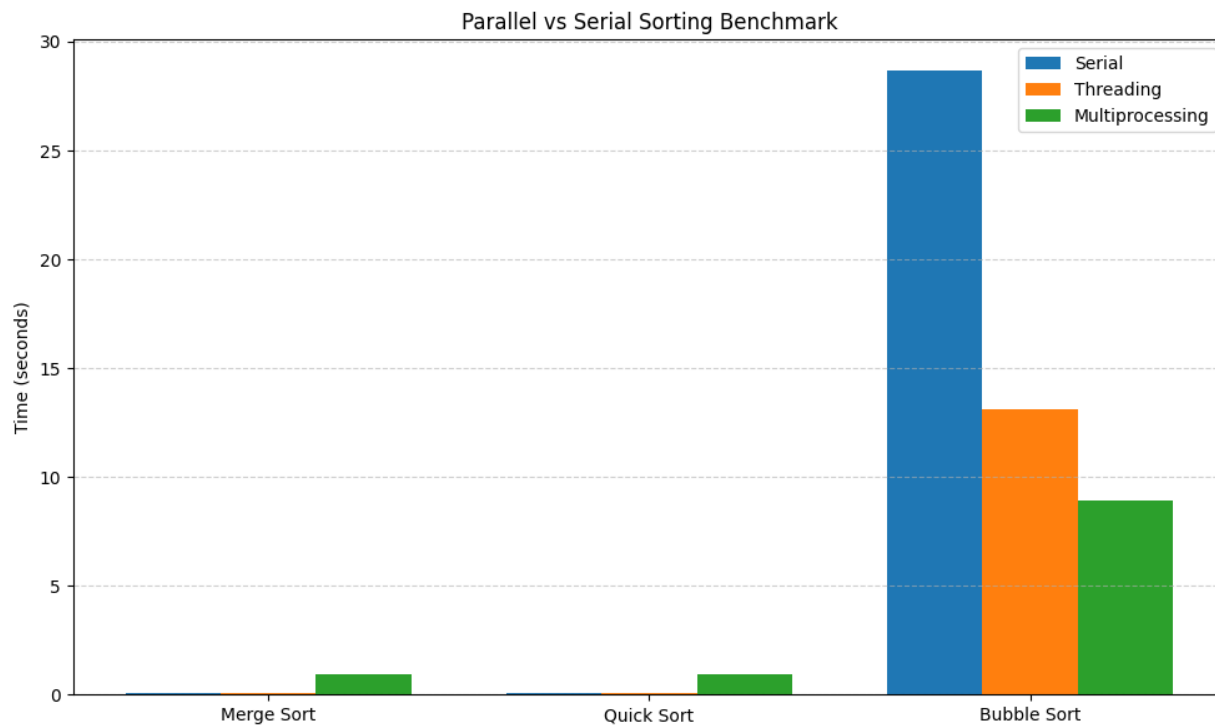
28. Визуелен приказ од резултатот при мерењето на сет од 15 000 податоци

```

© results.json > ...
1  {
2    "Merge Sort": [
3      0.07088994979858398,
4      0.06807446479797363,
5      1.1293375492095947
6    ],
7    "Quick Sort": [
8      0.05707669258117676,
9      0.06397557258605957,
10     1.0412421226501465
11   ],
12   "Bubble Sort": [
13     24.37988519668579,
14     14.832612037658691,
15     10.573016881942749
16   ]
17 }

```

29. Резултати при мерењето на сет од 20 000 податоци



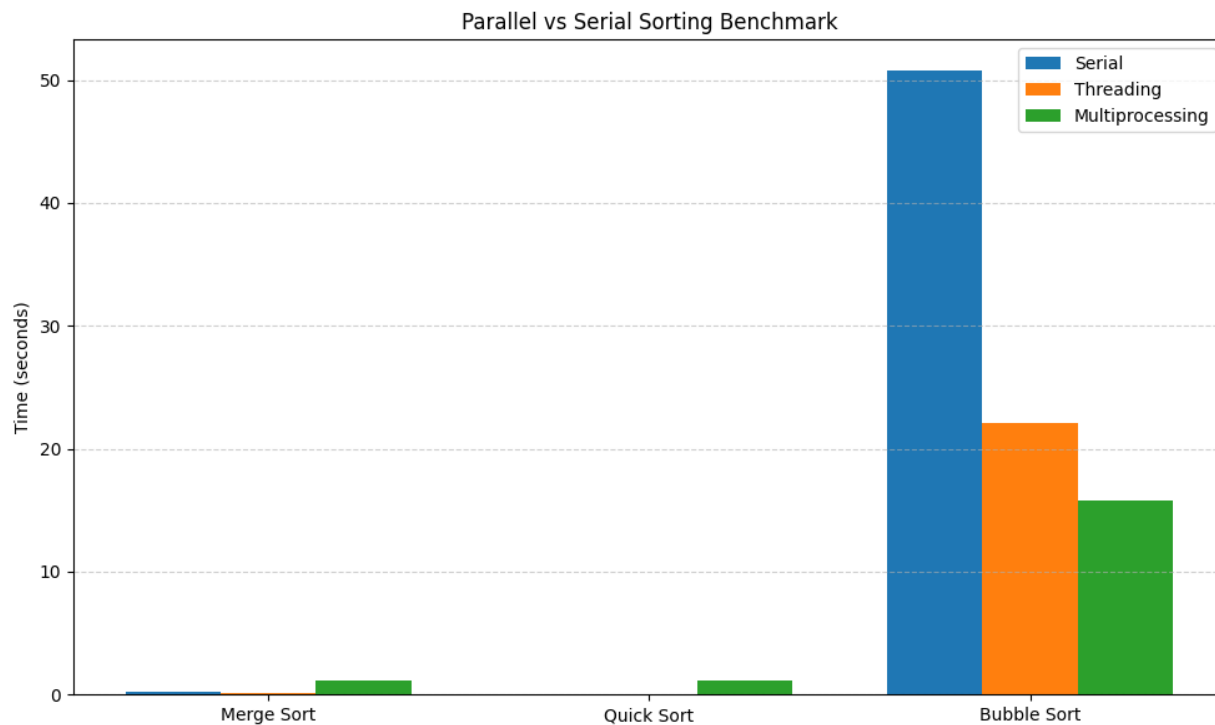
30. Визуелен приказ од резултатот при мерењето на сет од 20 000 податоци

```

results.json > [ ] Bubble Sort
1  {
2    "Merge Sort": [
3      0.12356138229370117,
4      0.16456151008605957,
5      1.0007853507995605
6    ],
7    "Quick Sort": [
8      0.07835078239440918,
9      0.0705416202545166,
10     0.9448695182800293
11   ],
12   "Bubble Sort": [
13     47.67263746261597,
14     22.49091148376465,
15     15.082140445709229
16   ]
17 }

```

31. Резултати при мерењето на сет од 25 000 податоци



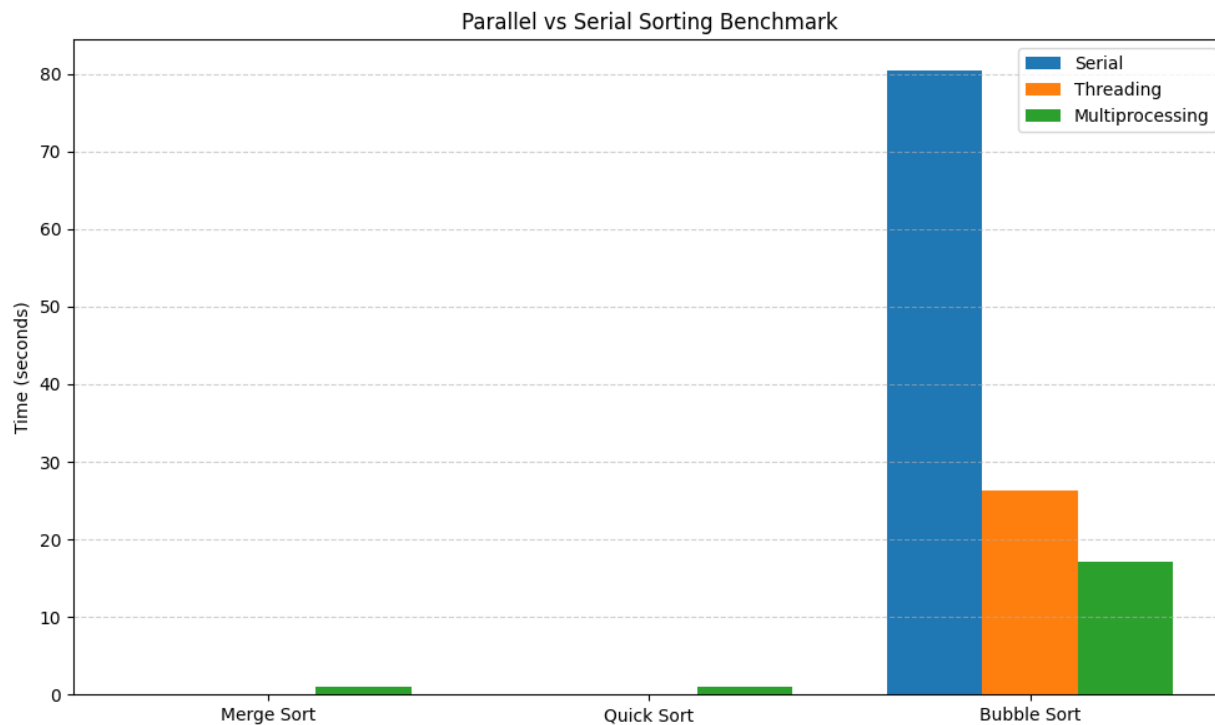
32. Визуелен приказ од резултатот при мерењето на сет од 25 000 податоци

```

© results.json > ...
1  {
2    "Merge Sort": [
3      0.0882875919342041,
4      0.09657859802246094,
5      0.928372859954834
6    ],
7    "Quick Sort": [
8      0.09565520286560059,
9      0.07950210571289062,
10     0.953383207321167
11   ],
12   "Bubble Sort": [
13     55.259621143341064,
14     26.725477933883667,
15     19.716119289398193
16   ]
17 }

```

33. Резултати при мерењето на сет од 30 000 податоци



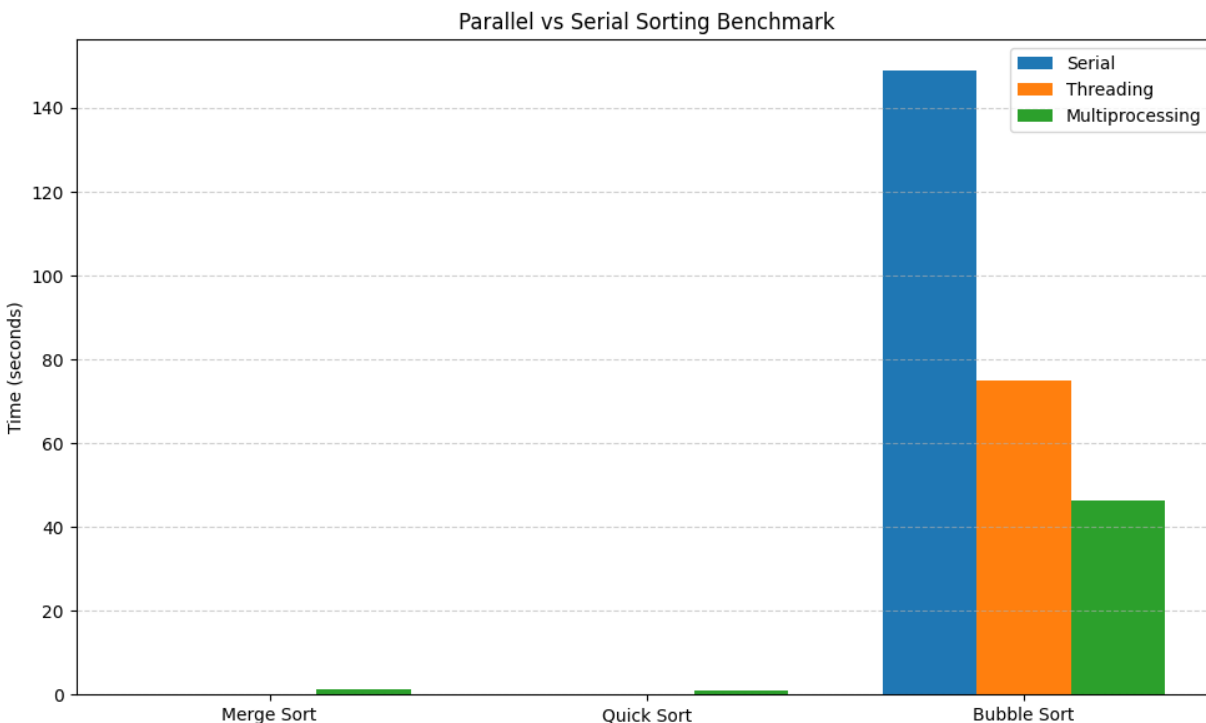
34. Визуелен приказ од резултатот при мерењето на сет од 30 000 податоци

```

@ results.json > ...
1  {
2    "Merge Sort": [
3      0.26006221771240234,
4      0.2151775360107422,
5      1.2600963115692139
6    ],
7    "Quick Sort": [
8      0.13495635986328125,
9      0.14760422706604004,
10     1.0326435565948486
11   ],
12   "Bubble Sort": [
13     183.31368923187256,
14     81.47094821929932,
15     49.83393430709839
16   ]
17 }

```

35. Резултати при мерењето на сет од 50 000 податоци



36. Визуелен приказ од резултатот при мерењето на сет од 50 000 податоци

Сетот од 50 000 елементи е праг-точка во експериментот.

Поради екстремната бавност, *Bubble Sort* е исклучен од понатамошните тестови. Неговата квадратична сложеност го прави непрактичен за понатамошна скалабилна анализа.

Merge Sort multiprocessing верзијата покажува јасно забрзување. Забрзувањето значително расте, а ефикасноста се стабилизира бидејќи секој процес има доволно пресметковна работа.

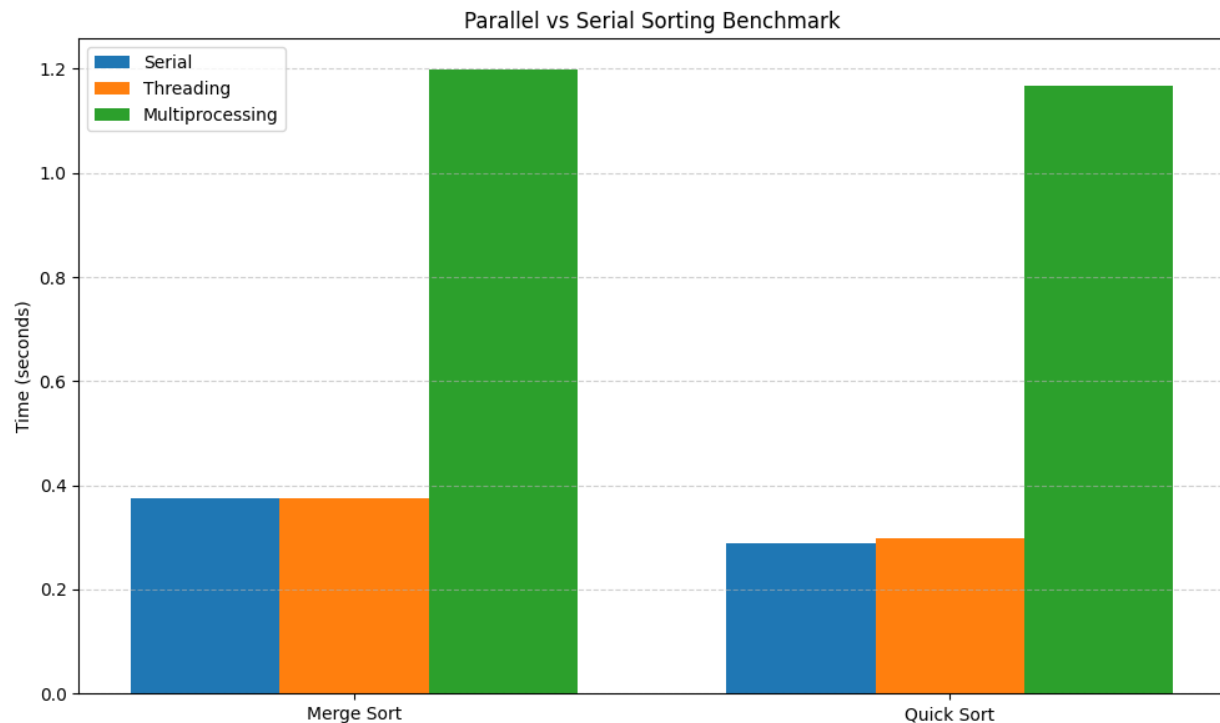
Quick Sort multiprocessing овде често го дава најдоброто време. Забрзувањето е високо, а ефикасноста е блиска до оптималната кога поделбата е рамномерна.

Во сетот од 1 000 000 елементи паралелизацијата целосно доаѓа до израз.

При ***Merge Sort*** забрзувањето станува значајно. Процесите работат паралелно врз големи подлисти. Ефикасноста расте бидејќи overhead е занемарлив во однос на пресметката.

Quick Sort често го надминува **Merge Sort** поради ниските мемориски трошоци. Забрзувањето е високо, но зависи од распределбата на pivot.

Threading кај двата алгоритми и понатаму е ограничен од *GIL* и не покажува вистинско забрзување.

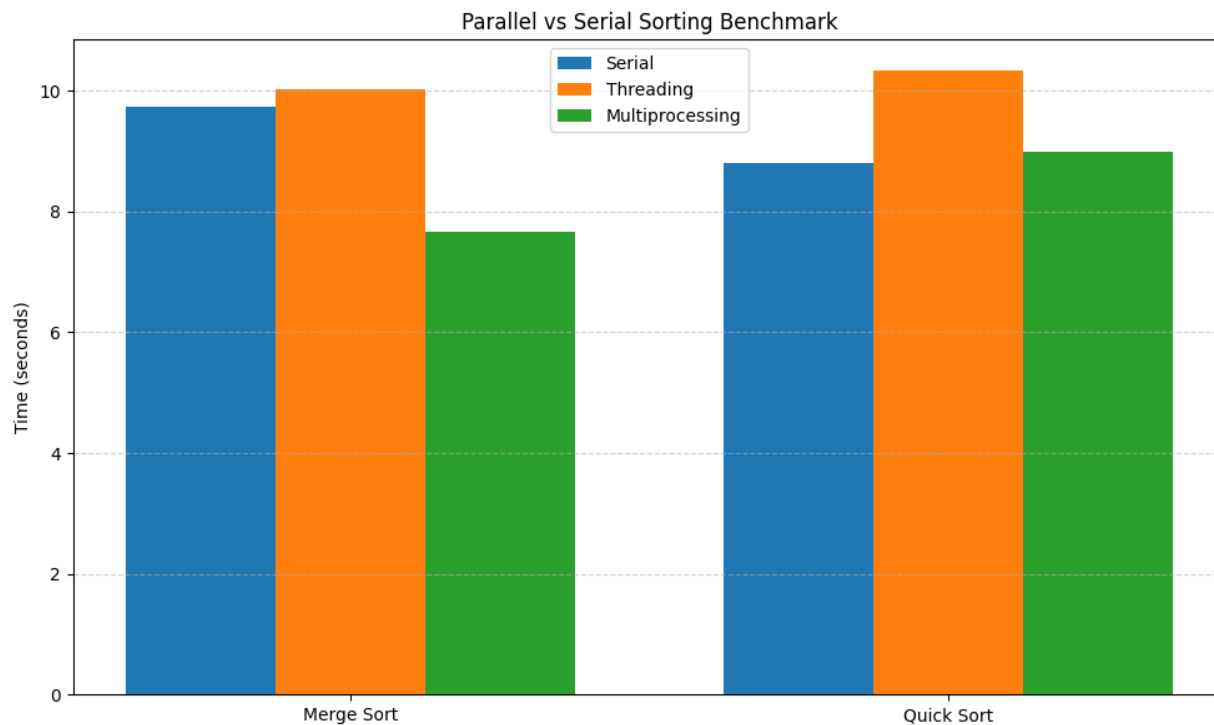


37. Визуелен приказ од резултатот при мерењето на сет од 1 000 000 податоци

Со сет од 2 000 000 елементи се постигнува паралелна скалабилност.

Забрзувањето на **Merge Sort** продолжува да расте, но со намалена стапка. Причината е што merge фазата останува сериска компонента, што го ограничува максималното забрзување (Amdahl ефект). Ефикасноста останува висока бидејќи процесите се добро искористени.

Quick Sort покажува високо забрзување, често поголемо од **Merge Sort**. Ефикасноста е висока кога партиционирањето е балансирано.

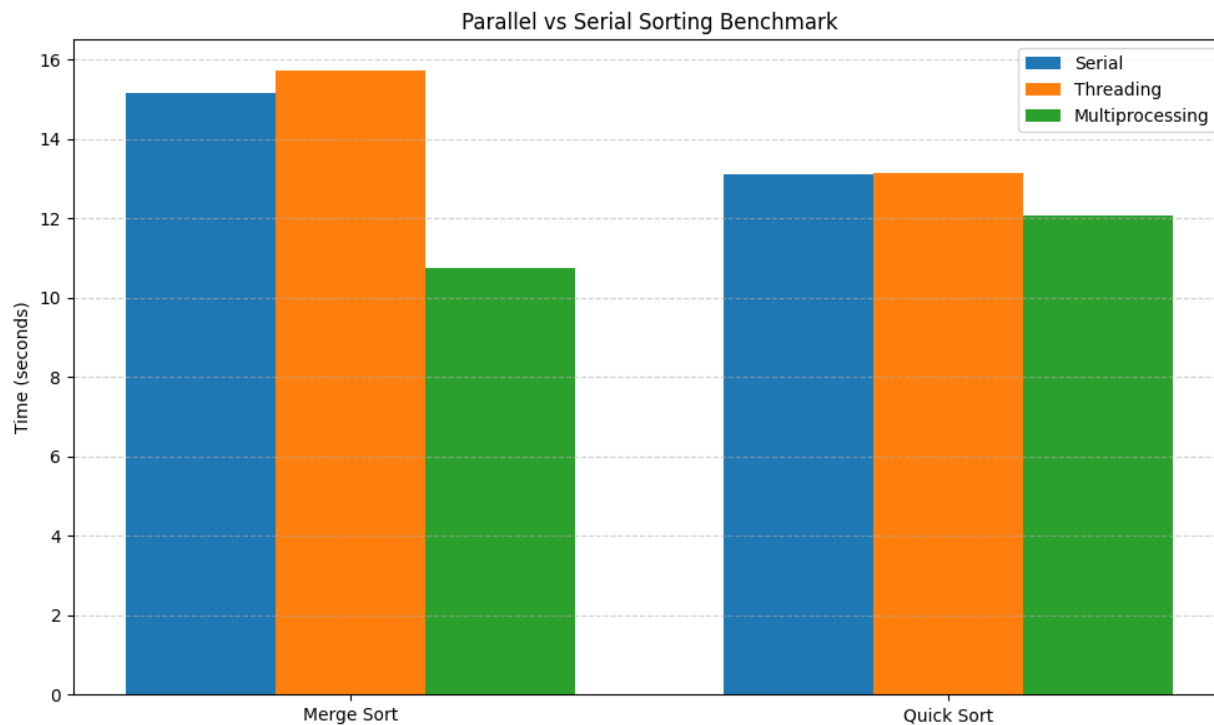


38. Визуелен приказ од резултатот при мерењето на сет од 2 000 000 податоци

Во следната фаза се појавуваат и системски ограничувања.

Копирањето на меморијата и интерпроцесната комуникација почнуваат да влијаат. Забрзувањето расте, но ефикасноста благо опаѓа поради зголемен комуникациски overhead.

Quick Sort и понатаму има силни перформанси. Поради in-place партиционирањето, меморискиот overhead е помал од *Merge Sort*, што ја одржува ефикасноста повисока.

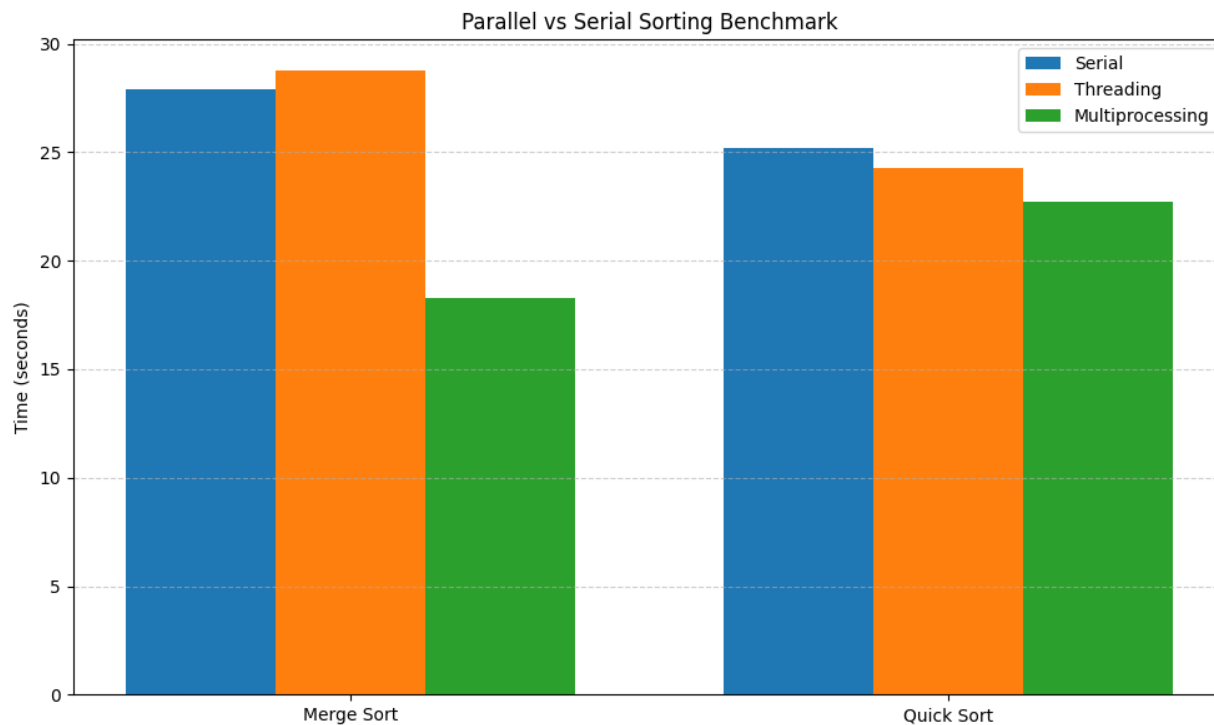


39. Визуелен приказ од резултатот при мерењето на сет од 3 000 000 податоци

Максималниот тест обем е најдобар показател за скалабилност.

Merge Sort покажува стабилно, но ограничено забрзување. Merge фазата, која е сериска, станува доминантен bottleneck. Ефикасноста опаѓа, но останува значајна.

Quick Sort multiprocessing дава најдобри резултати. Забрзувањето е највисоко во целиот експеримент. Ефикасноста останува релативно висока поради добрата распределба на работата и понизок комуникациски трошок.



40. Визуелен приказ од резултатот при мерењето на сет од 5 000 000 податоци

Експерименталната анализа јасно покажува дека паралелното сортирање не нуди универзално забрзување, туку неговата ефикасност е условена од интеракцијата меѓу:

- Алгоритамската сложеност
- Грануларноста на задачата
- Трошокот на комуникација
- Балансот на подзадачите

Bubble Sort останува демонстративен модел на лоша паралелна скалабилност, **Merge Sort** претставува стабилен и предвидлив паралелен алгоритам, додека **Quick Sort** нуди највисок потенцијал за забрзување, но со варијабилна ефикасност зависна од структурата на податоците.

7. Проширување на експерименталната анализа на паралелното сортирање

Досегашната експериментална анализа ни покажа јасни разлики помеѓу сериското и паралелното сортирање врз основа на големината на податочниот сет. Сепак, анализата ограничена само на промената на големината на податочни сетови не е доволна за целосно разбирање на ефектите од паралелизацијата. За да го прошириме експериментот и да ја истражимо вистинската ефикасност на различните алгоритми, потребно е да внесеме дополнителни критериуми и метрики, кои ќе овозможат поточна и објективна проценка на перформансите.

Во продолжение, официјално ќе ги воведеме следните дефиниции и формули:

1. Забрзување (*Speedup, S*):

Забрзувањето е мерка која покажува колку пати е побрзо извршувањето на алгоритмот кога се користи паралелна обработка во однос на сериско извршување. Се дефинира како:

$$S = \frac{T_{serial}}{T_{parallel}}$$

Каде T_{serial} е време на извршување на сериската верзија на алгоритмот, а $T_{parallel}$ е времето на извршување на паралелната верзија.

2. Ефикасност (*Efficiency, E*):

Ефикасноста ја мери искористеноста на процесорите во паралелното извршување. Таа се дефинира како забрзувањето поделено со бројот на процесори p :

$$E = \frac{S}{p} = \frac{T_{serial}}{p \cdot T_{parallel}}$$

Високата ефикасност означува добро распределена работа меѓу процесорите и минимални губитоци од комуникациски и синхронизациски трошоци.

3. Балансираност на задачите:

Балансираноста се оценува преку споредба на времето што секој процесор го троши на пресметка. Ако деловите се нерамномерно поделени, некои процесори ќе мируваат, додека другите работат, што го намалува вкупното забрзување.

4. *Amdahl* –ов закон:

Amdahl – ов закон ја ограничува максималната вредност на забрзувањето врз основа на делот од алгоритмот кој не може да се паралелизира. Ако f е делот кој е секвенцијален:

$$S_{max} = \frac{1}{f + \frac{1-f}{p}}$$

Оваа формула објаснува зошто дури и кај големи податочни сетови не се постигнува линеарно забрзување.

5. *Gustavson* –ов закон:

Gustavson –ов закон, пак, ја опишува скалабилноста кога големината на задачата расте заедно со бројот на процесори, овозможувајќи поголемо забрзување од она предвидено со *Amdahl* – ов закон, бидејќи поголемиот број на податоци овозможува подобра искористеност на процесорите.

Со овие дефиниции, експериментот беше проширен преку нова програма, која освен сериско, *threading* и *multiprocessing* мерења на времето, пресметува и забрзување, ефикасност и балансираност на задачите. Дополнително, програмата генерира визуелизации кои ги покажуваат зависностите помеѓу големината на сетот, бројот на процеси и перформансите на алгоритмите.

Ова продолжување овозможува поцелосна анализа на паралелното сортирање: се следат не само апсолутните времиња, туку и динамиката на процесорската ефикасност, ефектите на балансот на подзадачи, како и ограничувањата кои ги наметнува *Amdahl* – ов и *Gustavson* –ов закон. На овој начин се добива реална слика за тоа кога паралелизацијата е ефективна и кои алгоритми нудат најголем потенцијал за забрзување, во зависност од структурата на податоците и грануларноста на задачите.

7.1. Компаративна анализа на имплементацијата на алгоритмите

Во рамките на проектот беа реализирани две различни имплементации на паралелни сортички алгоритми, кои иако функционално извршуваат исти операции – сериско, *threading* и *multiprocessing* сортирање на *Merge Sort*, *Quick Sort* и *Bubble Sort* – суштински се разликуваат во својата архитектура, филозофија и степен на скалабилност. Анализата на овие разлики овозможува подлабоко разбирање на паралелниот модел, како и на ефектите што различните дизајнерски одлуки ги имаат врз перформансите.

Првата имплементација е конципирана како демонстративен модел на паралелизација. Кај неа паралелизмот е ограничен на едно ниво, при што листата секогаш се дели на две половини, а обработката се извршува со точно две нишки или два процеса. По иницијалната поделба, рекурзивното сортирање продолжува сериски. Овој пристап резултира со плиток паралелизам, односно паралелниот дел од алгоритмот е релативно мал во однос на целокупната работа. Таквата структура природно го ограничува максималното можно забрзување, бидејќи според *Amdahl* – овиот закон, колку е поголем серискиот дел од алгоритмот, толку е помал потенцијалот за скалабилност. Сепак, оваа имплементација има важна предност: поради ограничениот број на извршувачки единици, комуникациските и синхронизациските трошоци се минимизирани, што ја прави стабилна и ефикасна за помали и средни податочни сетови.

Втората имплементација претставува значително понапреден и пофлексибилен пристап. Наместо фиксна поделба на две половини, листата се дели на произволен број делови, во зависност од бројот на нишки или процеси што ги задава корисникот. Со тоа грануларноста на задачата станува динамичка, а паралелниот модел се прилагодува на конкретната експериментална поставка. Воведувањето на општи функции кои примаат “основен алгоритам” како параметар овозможува унифицирана архитектура и елиминира дуплирање на кодот, со што системот станува поелегантен и истражувачки ориентиран.

```
def split_list(arr, parts):
    if parts <= 1:
        return [arr]
    chunk_size = len(arr) // parts
    chunks = []
    for i in range(parts):
        start = i * chunk_size
        end = None if i == parts - 1 else (i + 1) * chunk_size
        chunks.append(arr[start:end])
    return chunks
```

41. Функција split_list()

Оваа промена има значајни последици врз перформансите. Бидејќи поголем дел од работата може да се извршува паралелно, паралелниот дел од алгоритмот се зголемува, што теоретски овозможува поголемо забрзување. Според *Gustavson* – овиот закон, при зголемување на големината на проблемот заедно со бројот на процесори, скалабилноста може да биде значително подобра отколку што предвидува *Amdahl* – овиот модел. Во практиката, тоа значи дека втората имплементација покажува поголем потенцијал кај големи податочни сетови, особено кога се користат повеќе процесори.

Сепак, зголемената флексибилност носи и дополнителни трошоци. Делбата на повеќе сегменти може да доведе до нерамномерна распределба на податоците, особено ако должината на листата не е деллива со бројот на процесори. Во такви случаи се јавува одредено нарушување на балансираноста, при што некои извршувачки единици завршуваат порано од други. Дополнително, секоја паралелна поделба и последователно спојување воведува комуникациски overhead, кој кај мали податочни сетови може дури и да ја намали ефикасноста. Оттука, втората имплементација е почувствителна на големината на проблемот и на изборот на бројот на процеси, но токму таа чувствителност овозможува подлабока експериментална анализа.

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

42. Функција merge()

Во однос на забрзувањето и ефикасноста, првата верзија е природно ограничена на максимум од две извршувачки единици, што значи дека нејзиниот *Speedup* не може да расте над таа граница. Од друга страна, втората верзија овозможува системско испитување на зависноста меѓу бројот на процеси и добиеното забрзување. При зголемување на бројот на процеси, забрзувањето може да расте, но ефикасноста постепено опаѓа поради зголемените трошоци за комуникација и синхронизација. Ова овозможува реална демонстрација на теоретските модели за паралелизам и нивната примена во практична средина.

Суштински, разликата меѓу двете програми не е само техничка, туку концептуална. Првата имплементација служи како чиста демонстрација на основните принципи на паралелното програмирање со минимален ризик од комплексност и нестабилност. Втората имплементација, напротив, претставува истражувачка платформа која овозможува анализа на скалабилност, балансираност, ефикасност и реално однесување на алгоритмите при различни конфигурации. Таа ја трансформира задачата од едноставна демонстрација во системско експериментално испитување на перформансите на паралелните сортирачки алгоритми.

Со тоа, преминот од првата кон втората верзија претставува еволуција од статички модел на паралелизам кон динамички и параметарски управуван систем,

кој овозможува подлабоко разбирање на односот меѓу теоријата и практиката во паралелното програмирање.

```
def merge_sort_serial(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort_serial(arr[:mid])
    right = merge_sort_serial(arr[mid:])
    return merge(left, right)

def quick_sort_serial(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort_serial(left) + mid + quick_sort_serial(right)

def bubble_sort_serial(arr):
    arr = arr.copy()
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
```

43. Функции за сериско извршување на трите алгоритми


```

def threaded_sort(arr, num_threads, base_sort):
    if num_threads <= 1:
        return base_sort(arr)
    chunks = split_list(arr, num_threads)
    results = [None] * len(chunks)
    threads = []

    def worker(i):
        results[i] = base_sort(chunks[i])

    for i in range(len(chunks)):
        t = threading.Thread(target=worker, args=(i,))
        threads.append(t)
        t.start()

    for t in threads:
        t.join()

    while len(results) > 1:
        new_results = []
        for i in range(0, len(results), 2):
            if i + 1 < len(results):
                new_results.append(merge(results[i], results[i + 1]))
            else:
                new_results.append(results[i])
        results = new_results
    return results[0]

```

44. Функција за извршување на паралелни нишки

```

def process_sort(arr, num_processes, base_sort):
    if num_processes <= 1:
        return base_sort(arr)
    chunks = split_list(arr, num_processes)
    with multiprocessing.Pool(processes=num_processes) as pool:
        results = pool.map(base_sort, chunks)
    while len(results) > 1:
        new_results = []
        for i in range(0, len(results), 2):
            if i + 1 < len(results):
                new_results.append(merge(results[i], results[i + 1]))
            else:
                new_results.append(results[i])
        results = new_results
    return results[0]

```

45. Функција за извршување на паралелни процеси

На сликите 43, 44 и 45 може да ја видиме сериската имплементација на алгоритмите и алгоритми за извршување на нишките/процесите според динамички зададен број на нишки/процеси (*num_threads/num_processes*), големина на низата (*arr*) и апстракција за алгоритмот (*base_sort*), што носи подобрена модуларност, експериментална контрола и системска анализа на скалабилност.

И на крај, wrappers, кои ја поврзуваат динамичката архитектура со конкретен алгоритам:

```
def merge_sort_threading(arr, threads):
    return threaded_sort(arr, threads, merge_sort_serial)

def merge_sort_multiprocessing(arr, processes):
    return process_sort(arr, processes, merge_sort_serial)

def quick_sort_threading(arr, threads):
    return threaded_sort(arr, threads, quick_sort_serial)

def quick_sort_multiprocessing(arr, processes):
    return process_sort(arr, processes, quick_sort_serial)

def bubble_sort_threading(arr, threads):
    return threaded_sort(arr, threads, bubble_sort_serial)

def bubble_sort_multiprocessing(arr, processes):
    return process_sort(arr, processes, bubble_sort_serial)
```

46. Wrappers

7.2. Анализа на експерименталниот и мерниот механизам

По компаративната анализа на имплементацијата на алгоритмите, следниот логичен чекор во истражувањето претставува анализа на системот за мерење и евалуација на нивните перформанси. Додека претходното поглавје беше насочено кон архитектонските разлики и паралелната структура на извршување, ова поглавје го разгледува модулот задолжен за организирање, спроведување и квантитативна обработка на експериментите. Во тој контекст

benchmark_engine.py претставува централен експериментален механизам преку кој теоретските концепти на паралелното програмирање се трансформираат во мерливи и анализирани резултати.

```
import time
import random
from sorting_algorithms import *

# Constants
DATASETS = [5, 5000, 10000, 20000, 50000, 100000, 500000, 1000000, 3000000, 5000000]
WORKERS = [2, 3, 4, 5, 6, 7, 8]
MAX_BUBBLE = 50000
THREAD_LIMIT = 1000000
MAX_WORKERS = 8

ALGORITHMS = {
    "Merge": (merge_sort_serial, merge_sort_threading, merge_sort_multiprocessing),
    "Quick": (quick_sort_serial, quick_sort_threading, quick_sort_multiprocessing),
    "Bubble": (bubble_sort_serial, bubble_sort_threading, bubble_sort_multiprocessing)
}
```

47. Задавање на константи за параметрите

Овој модул ја има улогата на посредник меѓу алгоритамската имплементација и аналитичкиот дел. Тој обезбедува контролирана средина во која алгоритмите се извршуваат под однапред дефинирани услови. Се дефинирање на различни големини на податочни сетови, различен број на работни единици и повеќе режими на извршување, системот создава структуриран експериментален простор во кој може да се испитува однесувањето на алгоритмите во сериски и паралелен контекст.

Дефинираните големини на податочни сетови овозможуваат анализа на скалабилноста во широк опсег – од минимални вредности до повеќемилионски елементи. Ова е од суштинско значење за проверка на однесувањето на алгоритмите при зголемување на големината на проблемот, што директно се поврзува со принципите на силен и слаб скалабилитет. На овој начин се овозможува практичната проверка на теоретските модели како *Amdahl* – овиот и *Gustavson* – овиот закон, кои ја опишуваат зависноста меѓу степенот на паралелизација и големината на проблемот.

Посебно внимание е посветено на контролата на бројот на извршувачки единици. Со дефинирање на опсег на работници (нишки и процеси), модулот овозможува системско испитување на зависноста меѓу бројот на процесори и добиеното забрзување. Овој аспект е клучен за анализа на saturation point, односно моментот кога зголемувањето на бројот на работници повеќе не резултира со пропорционално подобрување на перформансите. Токму тука станува видлив ефектот на комуникацискиот overhead и синхронизацијата, кои постепено ја намалуваат ефикасноста.

Функцијата за мерење на времето на извршување претставува основен инструмент за пресметка на перформансите. Со мерење на реалното време потребно за извршување на сериската и паралелната верзија, се создава основа за пресметка на забрзувањето. Односот меѓу сериското и паралелното време директно го дефинира *Speedup* факторот, кој претставува централна метрика во анализа на паралелни системи. Овој пристап овозможува објективна споредба меѓу различните модели на извршување и алгоритми.

Покрај мерењето на времето, модулот имплементира и пресметка на балансираноста на оптоварувањето. Преку симулација на распределбата на податоците во сегменти, се пресметува односот меѓу просечната и максималната големина на chunks. Доколку сите сегменти се еднакви, балансираноста достигнува 100%, што укажува на оптимална распределба на работата. Овој показател е особено важен бидејќи нерамномерната распределба може да доведе до ситуација во која некои процеси завршуваат порано, додека други продолжуваат да работат, што ја намалува целокупната ефикасност на системот.

```

def generate_dataset(size):
    return [random.randint(1, 100000) for _ in range(size)]

def benchmark(func, data, workers=None):
    start = time.time()
    if workers is None:
        func(data.copy())
    else:
        func(data.copy(), workers)
    return time.time() - start

def calculate_load_efficiency(data_size, workers):
    if workers <= 1:
        return 100.0, [data_size]
    chunk_size = data_size // workers
    remainder = data_size % workers
    chunks = [chunk_size] * workers
    chunks[-1] += remainder
    avg = sum(chunks) / workers
    efficiency = (avg / max(chunks)) * 100
    return efficiency, chunks

```

48. Функции за мерење на времето на извршување и балансираноста за алгоритмите

Модулот овозможува три различни режими на експериментално извршување. Основниот режим овозможува фиксна и контролирана споредба меѓу серискиот, `threading` и `multiprocessing` моделот со ограничен број работници. Овој режим служи како почетна референтна точка.

```

def run_basic_benchmark(dataset_size, selected_algorithms):
    results = []
    data = generate_dataset(dataset_size)
    workers = 2

    for algo_name in selected_algorithms:
        if algo_name == "Bubble" and dataset_size > MAX_BUBBLE:
            continue

        serial_func, thread_func, process_func = ALGORITHMS[algo_name]

        s_time = benchmark(serial_func, data.copy())
        t_time = benchmark(thread_func, data.copy(), workers) if dataset_size <= THREAD_LIMIT else None
        p_time = benchmark(process_func, data.copy(), workers)

        thread_speedup = s_time / t_time if t_time else None
        process_speedup = s_time / p_time

        load_eff, chunks = calculate_load_efficiency(dataset_size, workers)

        results.append({
            "algorithm": algo_name,
            "serial_time": s_time,
            "thread_time": t_time,
            "process_time": p_time,
            "thread_speedup": thread_speedup,
            "process_speedup": process_speedup,
            "load_efficiency": load_eff,
            "chunks": chunks,
            "workers": workers,
            "dataset": dataset_size
        })

    return results

```

49. Мерења кои се извршуваат во основен режим

Корисничкиот режим овозможува параметарска флексибилност, при што корисникот може самостојно да го избере алгоритмот, големината на податочниот сет и бројот на работници. Со тоа се создава можност за таргетирана анализа на специфични сценарија.

```

def run_user_benchmark(dataset_size, algorithm_name, exec_mode, num_workers):
    results = []
    data = generate_dataset(dataset_size)

    if algorithm_name not in ALGORITHMS:
        return results

    serial_func, thread_func, process_func = ALGORITHMS[algorithm_name]

    if algorithm_name == "Bubble" and dataset_size > MAX_BUBBLE:
        return results

    s_time = benchmark(serial_func, data.copy())

    if exec_mode in ["all", "serial"]:
        load_eff, chunks = calculate_load_efficiency(dataset_size, 1)
        results.append({
            "algorithm": f"{algorithm_name}(serial)",
            "serial_time": s_time,
            "thread_time": None,
            "process_time": None,
            "thread_speedup": None,
            "process_speedup": None,
            "load_efficiency": 100.0,
            "chunks": [dataset_size],
            "workers": 1,
            "dataset": dataset_size,
            "exec_type": "serial"
        })

```

```

if exec_mode in ["all", "thread"] and dataset_size <= THREAD_LIMIT:
    t_time = benchmark(thread_func, data.copy(), num_workers)
    thread_speedup = s_time / t_time
    load_eff, chunks = calculate_load_efficiency(dataset_size, num_workers)
    results.append({
        "algorithm": f"{algorithm_name}(thread)",
        "serial_time": s_time,
        "thread_time": t_time,
        "process_time": None,
        "thread_speedup": thread_speedup,
        "process_speedup": None,
        "load_efficiency": load_eff,
        "chunks": chunks,
        "workers": num_workers,
        "dataset": dataset_size,
        "exec_type": "thread"
    })

if exec_mode in ["all", "process"]:
    p_time = benchmark(process_func, data.copy(), num_workers)
    process_speedup = s_time / p_time
    load_eff, chunks = calculate_load_efficiency(dataset_size, num_workers)
    results.append({
        "algorithm": f"{algorithm_name}(process)",
        "serial_time": s_time,
        "thread_time": None,
        "process_time": p_time,
        "thread_speedup": None,
        "process_speedup": process_speedup,
        "load_efficiency": load_eff,
        "chunks": chunks,
        "workers": num_workers,
        "dataset": dataset_size,
        "exec_type": "process"
    })

return results

```

50. Мерења кои се звршуваат во кориснички режим

Автоматскиот режим, пак, претставува целосна експериментална итерација низ сите можни комбинации на параметри, генерирајќи обемна база на податоци погодна за статистичка и визуелна анализа.


```

def run_auto_benchmark():
    results = []

    for size in DATASETS:
        data = generate_dataset(size)

        for algo_name in ["Merge", "Quick", "Bubble"]:
            if algo_name == "Bubble" and size > MAX_BUBBLE:
                continue

            serial_func, thread_func, process_func = ALGORITHMS[algo_name]

            s_time = benchmark(serial_func, data.copy())

            for workers in WORKERS:
                if size <= THREAD_LIMIT:
                    t_time = benchmark(thread_func, data.copy(), workers)
                    thread_speedup = s_time / t_time
                else:
                    t_time = None
                    thread_speedup = None

                p_time = benchmark(process_func, data.copy(), workers)
                process_speedup = s_time / p_time
                load_eff, chunks = calculate_load_efficiency(size, workers)

                results.append({
                    "dataset": size,
                    "workers": workers,
                    "algorithm": algo_name,
                    "serial_time": s_time,
                    "thread_time": t_time,
                    "process_time": p_time,
                    "thread_speedup": thread_speedup,
                    "process_speedup": process_speedup,
                    "load_efficiency": load_eff,
                    "chunks": chunks
                })

    return results

```

51. Мерења кои се извршуваат во автоматски режим

Особено значајно е тоа што резултатите од секој експеримент се чуваат во структуриран формат кој содржи информации за времињата на извршување, забрзувањето, балансираноста, бројот на работници и големината на податочниот сет. Ова овозможува понатамошна обработка, графичка визуелизација и

генерирање на *heatmap* анализи, а со тоа системот не само што мери перформанси, туку создава аналитичка основа за интерпретација и заклучување.

Во целина, модулот за мерење претставува експериментална инфраструктура која ги поврзува теоретските принципи на паралелното програмирање со нивната практична верификација. Тој претставува квантитативна анализа на забрзувањето, ефикасноста, скалабилноста и балансираноста, трансформирајќи ја програмата од едноставна демонстрација во системска истражувачка платформа. Со тоа се обезбедува научна валидност на добиените резултати и се создава основа за аргументирани заклучоци за перформансите на паралелните сортирачки алгоритми во рамките на дадениот проблем.

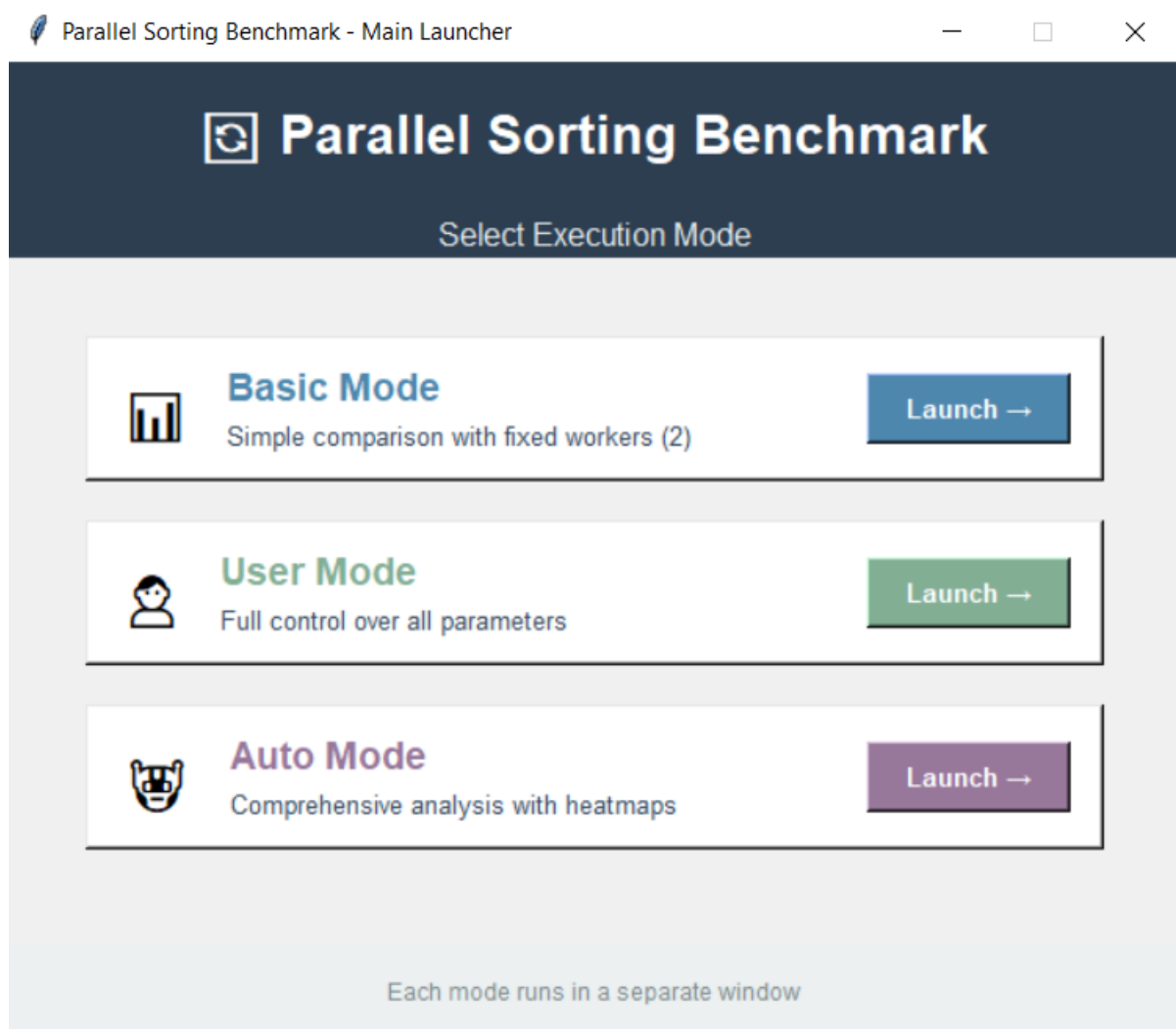
7.3. Графички кориснички интерфејс и визуелизација

Графичкиот кориснички интерфејс е реализиран преку три засебни фајлови, по еден за секој режим на работа: Basic, User и Auto. Ваквата организација овозможува модуларност и јасна поделба на функционалностите. Интерфејсот е изработен со библиотеката *tkinter*, со дополнителна употреба на *ttk* и *messagebox* за подобра визуелна структура и интеракција со корисникот. За извршување на benchmark процесите без блокирање на интерфејсот се користи *threading*, а за процесно паралелизирање *multiprocessing*. Поврзувањето со експерименталниот дел се реализира преку модулот *benchmark_engine.py*.

Визуелизацијата на резултатите е сместена во посебен фолдер со три фајлови, повторно според режимите на работа. За графичко прикажување се користат *matplotlib* и *seaborn*, додека *pandas* и *numpy* служат за обработка на податоците. Интеграцијата на графичите во интерфејсот се овозможува преку *FigureCanvasTkAgg*.

Во оваа семинарска работа интерфејсот и визуелниот приказ не се предмет на детална анализа, туку имаат поддржувачка улога во изведувањето и презентацијата на експериментите.

Извршувањето на програмата се иницира преку фајлот *main_launcher.py*, со командата *python main_launcher.py*.



52. Почетна форма на програмата

Сликата 52 ја прикажува почетната форма на програмата во која корисникот има можност да избере еден од прикажаните режими за извршување на паралелното сортирање. Со клик на копчето Launch на Basic Mode се отвара нова форма во која корисникот добива можност да избере еден или повеќе алгоритми за сортирање и големина на податочниот сет над кој ќе се врши сортирањето. Овој режим има фиксен број на нишки/процеси:

Basic Mode - Parallel Sorting Benchmark

Basic Mode

Fixed workers = 2

Parameters

Dataset Size: 5000
Algorithms: ☒ Merge Sort ☒ Quick Sort ☒ Bubble Sort

Note: Bubble sort skipped for size > 50,000

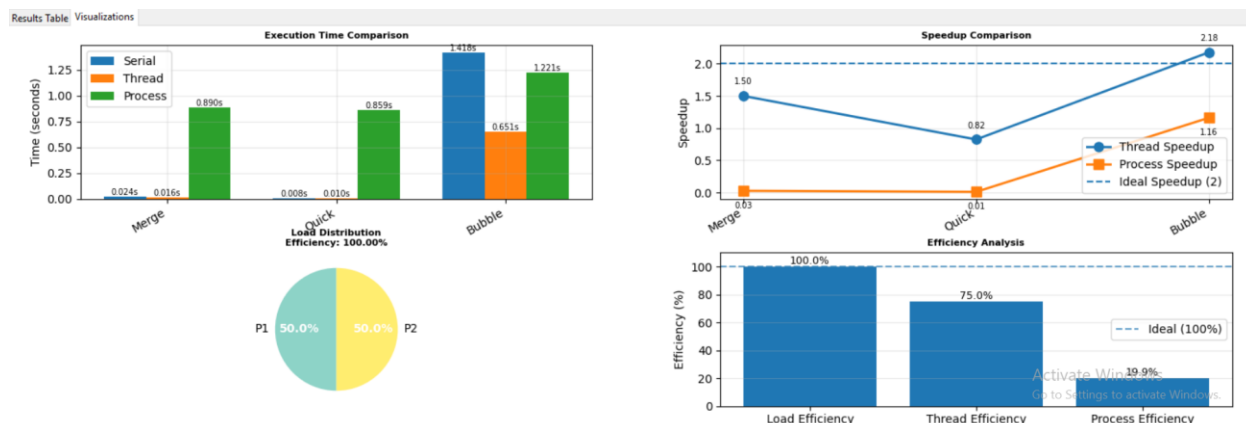
Run Benchmark

Results Table Visualizations

Algorithm	Serial(s)	Thread(s)	Process(s)	Thread Speedup	Process Speedup	Load Eff(%)	Chunks
Merge	0.0119	0.0124	0.9689	0.96	0.01	100.00	[2500, 2500]
Quick	0.0100	0.0137	0.9537	0.73	0.01	100.00	[2500, 2500]
Bubble	1.4728	0.7523	1.3183	1.96	1.12	100.00	[2500, 2500]

53. Графички кориснички интерфејс за основен режим

Со клик на копчето Run Benchmark се извршува селектираниот алгоритам над зададениот сет на податоци, притоа генерирајќи табеларни податоци за времињата на извршување, забрзувањето, балансираноста и распределбата на податоците меѓу нишките/процесите. Дополнително, корисникот има можност истите резултати да ги види визуелно со клик на табот Visualizations:



54. Визуелна репрезентација на резултатите за основен режим

Понатаму, со клик на копчето Launch на User Mode се отвара друга форма во која корисникот добива можност самостојно да ги избере сите параметри: големина на податочен сет, алгоритам за сортирање (Merge, Quick, Bubble), режим на извршување (serial, thread, process) и број на работници (нишки/процеси):

User Mode - Parallel Sorting Benchmark

User Mode

Full control over parameters

Parameters

Dataset Size: 10000

Algorithm: Bubble

Execution Mode: all

Number of Workers: 4

Note: Max workers = 8. Bubble sort skipped for size > 50,000

Run Benchmark

Results Table

Algorithm	Serial(s)	Thread(s)	Process(s)	Thread Speedup	Process Speedup	Load Eff(%)	Chunks
Bubble(serial)	5.4836	-	-	-	-	100.00	[10000]
Bubble(thread)	5.4836	0.5901	-	9.29	-	100.00	[1250, 1250, 1250, 125]
Bubble(process)	5.4836	-	2.0663	-	2.65	100.00	[1250, 1250, 1250, 125]

55. Графички кориснички интерфејс за кориснички режим

Со клик на копчето Run Benchmark се извршува селектираниот алгоритам со кориснички зададените параметри, при што се генерира табела со резултатите од времето на извршување, забрзувањето, балансираноста и распределбата на податоците меѓу процесите/нишките. Исто како и во претходниот режим, корисникот може да погледне и визуелна репрезентација на резултатите од извршувањето на сортирањето на алгоритмот според конкретните параметри:



56. Визуелна репрезентација на резултатите во кориснички режим

7.4. Автоматски режим на извршување

За целите на оваа семинарска работа, од најголемо значење е автоматскиот режим на извршување. Овој режим претставува централен експериментален механизам, бидејќи овозможува систематско и целосно тестирање на сите алгоритми, над сите дефинирани големини на податочни сетови, во сите режими на извршување (сериски, со нишки и со процеси) и со различен број на работници (нишки/процеси).

За разлика од останатите режими, кои служат за поединечни тестирања и демонстративна анализа, автоматскиот режим извршува комплетен експериментален циклус. Тој ги генерира сите можни комбинации на параметри и обезбедува унифицирано мерење на времето на извршување, вредностите на забрзување и ефикасноста. Со тоа се добива целосна слика за однесувањето на алгоритмите во различни услови на паралелизација.

Резултатите од автоматскиот режим се претставуваат преку визуелно прегледна табела што ги содржи сите измерени вредности, како и преку *heatmap* график кој овозможува интуитивна визуелна споредба на перформансите. *Heatmap* претставувањето овозможува лесно идентификување на областите со највисоки добивки во перформансите, како и оние комбинации каде паралелизацијата не носи значителен бенефит. Токму оваа визуелна анализа претставува клучен дел од експерименталниот дел, бидејќи овозможува брзо препознавање на трендовите и ограничувањата на поединечните алгоритми. Во семинарската работа овој дел ќе биде дополнително илустриран со слики од извршувањето.

Притоа, потребно е да се нагласат две експериментални ограничувања кои се воведени со цел да се обезбеди практичност и стабилност на тестирањата.

Алгоритмот *Bubble Sort* се извршува само за податочни сетови со максимална големина од 50 000 елементи. Причината за ова ограничување е неговата временска сложеност $O(n^2)$, која доведува до експоненцијално зголемување на времето на извршување при поголеми влезни податоци. За сетови со милион елементи, времето на извршување би било непропорционално долго и би го нарушило текот на експерименталната анализа.

Дополнително, извршувањето со нишки е исклучено за сетови поголеми од 1 000 000 елементи. Причината лежи во ограничувањата на GIL (Global Interpreter Lock) во Python, кој не дозволува вистинско паралелно извршување на CPU-bound задачи со нишки. Кај големи податочни сетови overhead —от од управувањето со нишките и синхронизацијата може да ја надмине потенцијалната добивка во перформанси, што резултира со неефикасно или дури и побавно извршување во споредба со серискиот режим.

Со овие контролирани ограничувања, автоматскиот режим обезбедува реална, стабилна и методолошки оправдана основа за компаративна анализа на паралелните алгоритми.

Во сетот на податоци е вклучен и мал пример со само 5 елеманти, со цел демонстрација на сценарија со лоша баласираност при распределба на задачите. Овој пример овозможува јасно да се визуелизира како паралелната обработка реагира на нерамномерно распределени податоци и кои фактори влијаат на ефикасноста и балансираноста.

Автоматскиот режим на извршување се активира со клик на копчето Launch на Auto Mode, при што се отвара нова форма во која корисникот е потребно да кликне на копчето Start Auto Analysis и извршувањето започнува со динамички и колоритно визуелен приказ:

Auto Mode - Parallel Sorting Benchmark

Auto Mode

Merge Quick Bubble

Start Auto Analysis

Dataset	Workers	Algorithm	Serial	Thread	Process	Thread Speedup	Process Speedup	Load Eff(%)
5	2	Merge	0.0	0.002	2.0104	-	-	83.33
5	3	Merge	0.0	0.002	2.1288	-	-	55.56
5	4	Merge	0.0	0.002	2.4527	-	-	62.5
5	5	Merge	0.0	0.0023	3.0682	-	-	100.0
5	6	Merge	0.0	0.002	3.7853	-	-	16.67
5	7	Merge	0.0	0.002	4.0324	-	-	14.29
5	8	Merge	0.0	0.004	4.5734	-	-	12.5
5	2	Quick	0.0	0.001	2.0829	-	-	83.33
5	3	Quick	0.0	0.001	2.4328	-	-	55.56
5	4	Quick	0.0	0.002	2.8344	-	-	62.5
5	5	Quick	0.0	0.002	3.2889	-	-	100.0
5	6	Quick	0.0	0.003	3.9221	-	-	16.67
5	7	Quick	0.0	0.003	4.4432	-	-	14.29
5	8	Quick	0.0	0.003	5.2488	-	-	12.5
5	2	Bubble	0.0	0.001	2.1918	-	-	83.33
5	3	Bubble	0.0	0.002	2.507	-	-	55.56
5	4	Bubble	0.0	0.003	2.9246	-	-	62.5

57. Графички кориснички интерфејс и визуелна репрезентација на резултатите за автоматски режим

7.5. Централна експериментална анализа

Во продолжение се разгледуваат резултатите од експерименталното извршување за секој податочен сет поединечно, при што анализата се фокусира на однесувањето на конкретните алгоритми, бројот на процеси и нишки, постигнатите забрзувања, степенот на балансираност на задачите и влијанието на системските ограничувања врз перформансите.

Dataset	Workers	Algorithm	Serial	Thread	Process	Thread Speedup	Process Speedup	Load Eff(%)
5	2	Merge	0.0	0.001	2.2991	-	-	83.33
5	3	Merge	0.0	0.002	2.2498	-	-	55.56
5	4	Merge	0.0	0.002	3.0933	-	-	62.5
5	5	Merge	0.0	0.0149	3.1902	-	-	100.0
5	6	Merge	0.0	0.002	3.5473	-	-	16.67
5	7	Merge	0.0	0.0037	4.1422	-	-	14.29
5	8	Merge	0.0	0.003	4.8256	-	-	12.5
5	2	Quick	0.0	0.001	2.1601	-	-	83.33
5	3	Quick	0.0	0.002	2.5176	-	-	55.56
5	4	Quick	0.0	0.003	3.0468	-	-	62.5
5	5	Quick	0.0	0.002	3.4806	-	-	100.0
5	6	Quick	0.0	0.003	3.9722	-	-	16.67
5	7	Quick	0.0	0.0035	4.563	-	-	14.29
5	8	Quick	0.0	0.0038	5.1858	-	-	12.5
5	2	Bubble	0.0	0.002	2.3485	-	-	83.33
5	3	Bubble	0.0	0.002	2.5722	-	-	55.56
5	4	Bubble	0.0	0.002	3.0691	-	-	62.5
5	5	Bubble	0.0	0.002	4.2311	-	-	100.0
5	6	Bubble	0.0	0.0022	4.4873	-	-	16.67
5	7	Bubble	0.0	0.003	5.1893	-	-	14.29
5	8	Bubble	0.0	0.0043	4.907	-	-	12.5

58. Резултати за податочен сет од 5 елементи

Кај малиот податочен сет од 5 елементи не се забележува никакво забрзување при паралелно извршување. Причината за тоа е доминантниот комуникациски overhead – времето потребно за креирање процеси или нишки, нивна иницијализација, распределба на задачи и синхронизација значително го надминува реалното време потребно за сортирање. Овој експеримент има демонстративна улога, односно ја илустрира важноста на правилната распределба на задачите. Единствено целосно балансирање се постигнува кога бројот на процесори е еднаков на бројот на елементи, при што секој процесор обработува по една задача. Сепак, и покрај совршената распределба, паралелното извршување не носи добивка поради несразмерниот системски трошок.

Dataset	Workers	Algorithm	Serial	Thread	Process	Thread Speedup	Process Speedup	Load Eff(%)
5000	2	Merge	0.0165	0.0192	3.0259	0.86	0.01	100.0
5000	3	Merge	0.0165	0.0506	2.871	0.33	0.01	99.92
5000	4	Merge	0.0165	0.0164	2.986	1.01	0.01	100.0
5000	5	Merge	0.0165	0.0325	4.1538	0.51	-	100.0
5000	6	Merge	0.0165	0.0196	4.322	0.85	-	99.8
5000	7	Merge	0.0165	0.0333	5.1455	0.5	-	99.76
5000	8	Merge	0.0165	0.0332	6.0372	0.5	-	100.0
5000	2	Quick	0.0123	0.013	2.4637	0.95	0.01	100.0
5000	3	Quick	0.0123	0.0341	2.8134	0.51	-	99.92
5000	4	Quick	0.0123	0.0258	3.0479	0.48	-	100.0
5000	5	Quick	0.0123	0.0244	3.5808	0.51	-	100.0
5000	6	Quick	0.0123	0.0189	3.8755	0.65	-	99.8
5000	7	Quick	0.0123	0.0162	4.408	0.76	-	99.76
5000	8	Quick	0.0123	0.0164	5.7594	0.75	-	100.0
5000	2	Bubble	1.6984	0.7168	2.6262	2.37	0.65	100.0
5000	3	Bubble	1.6984	0.4765	2.8195	3.56	0.6	99.92
5000	4	Bubble	1.6984	0.3652	3.1134	4.65	0.55	100.0
5000	5	Bubble	1.6984	0.2772	3.5136	6.13	0.48	100.0
5000	6	Bubble	1.6984	0.2271	3.9141	7.48	0.43	99.8
5000	7	Bubble	1.6984	0.2021	4.4694	8.4	0.38	99.76
5000	8	Bubble	1.6984	0.1753	4.8183	9.69	0.35	100.0

59. Резултати за податочен сет од 5 000 елементи

Кај податочниот сет од 5 000 елементи започнува да се забележува ефектот од паралелизацијата, но исклучиво кај алгоритмот Bubble Sort. Поради неговата квадратична временска сложеност $O(n^2)$, зголемувањето на големината на

податоците доведува до значително зголемување на извршниот товар, што овозможува подобра амортизација на overhead –от. Забрзувањето се движи од приближно 2x кај две нишки до над 9x кај осум нишки. За разлика од него, *Merge Sort* и *Quick Sort*, со сложеност $O(n \log n)$ и понатаму не покажуваат значајно подобрување, бидејќи нивното сериско извршување е доволно ефикасно и времетраењето е премало за паралелизацијата да донесе реална добивка.

Dataset	Workers	Algorithm	Serial	Thread	Process	Thread Speedup	Process Speedup	Load Eff(%)
10000	2	Merge	0.03	0.033	2.1948	0.91	0.01	100.0
10000	3	Merge	0.03	0.03	2.484	1.0	0.01	99.98
10000	4	Merge	0.03	0.0295	3.2451	1.02	0.01	100.0
10000	5	Merge	0.03	0.0335	4.1726	0.9	0.01	100.0
10000	6	Merge	0.03	0.0627	4.8704	0.48	0.01	99.8
10000	7	Merge	0.03	0.0352	4.7247	0.85	0.01	99.76
10000	8	Merge	0.03	0.0435	5.295	0.69	0.01	100.0
10000	2	Quick	0.0631	0.0947	3.6356	0.67	0.02	100.0
10000	3	Quick	0.0631	0.0583	2.8259	1.08	0.02	99.98
10000	4	Quick	0.0631	0.0284	3.5499	2.22	0.02	100.0
10000	5	Quick	0.0631	0.0674	4.2629	0.94	0.01	100.0
10000	6	Quick	0.0631	0.032	4.6482	1.97	0.01	99.8
10000	7	Quick	0.0631	0.0558	5.3813	1.13	0.01	99.76
10000	8	Quick	0.0631	0.0598	6.756	1.06	0.01	100.0
10000	2	Bubble	7.7222	3.8096	3.9137	2.03	1.97	100.0
10000	3	Bubble	7.7222	2.0198	3.4848	3.82	2.22	99.98
10000	4	Bubble	7.7222	1.5126	3.3347	5.11	2.32	100.0
10000	5	Bubble	7.7222	1.1574	4.4419	6.67	1.74	100.0
10000	6	Bubble	7.7222	0.9599	4.0054	8.04	1.93	99.8
10000	7	Bubble	7.7222	0.8392	4.6946	9.2	1.65	99.76
10000	8	Bubble	7.7222	0.6948	5.1275	11.11	1.51	100.0

60. Резултати за податочен сет од 10 000 елементи

Сличен тренд продолжува и кај сетот од 10 000 елементи. Се забележуваат минимални одобрувања кај *Merge Sort* и *Quick Sort*, но тие се статистички незначајни во однос на сериското време. *Bubble Sort* повторно покажува изразено забрзување, особено во имплементацијата со нишки.

Dataset	Workers	Algorithm	Serial	Thread	Process	Thread Speedup	Process Speedup	Load Eff(%)
20000	2	Merge	0.0751	0.0723	2.1789	1.04	0.03	100.0
20000	3	Merge	0.0751	0.0649	2.5033	1.16	0.03	99.98
20000	4	Merge	0.0751	0.0699	2.939	1.07	0.03	100.0
20000	5	Merge	0.0751	0.0645	3.3885	1.16	0.02	100.0
20000	6	Merge	0.0751	0.0811	5.0611	0.93	0.01	99.95
20000	7	Merge	0.0751	0.0862	5.3554	0.87	0.01	99.97
20000	8	Merge	0.0751	0.1188	5.4356	0.63	0.01	100.0
20000	2	Quick	0.0486	0.0943	2.2032	0.51	0.02	100.0
20000	3	Quick	0.0486	0.0757	3.1688	0.64	0.02	99.98
20000	4	Quick	0.0486	0.0811	3.7425	0.6	0.01	100.0
20000	5	Quick	0.0486	0.1054	4.0529	0.46	0.01	100.0
20000	6	Quick	0.0486	0.1203	4.6859	0.4	0.01	99.95
20000	7	Quick	0.0486	0.0696	4.3953	0.7	0.01	99.97
20000	8	Quick	0.0486	0.0624	4.9679	0.78	0.01	100.0
20000	2	Bubble	24.4411	11.1159	8.9437	2.2	2.73	100.0
20000	3	Bubble	24.4411	7.7556	5.8632	3.15	4.17	99.98
20000	4	Bubble	24.4411	5.8094	5.1413	4.21	4.75	100.0
20000	5	Bubble	24.4411	4.5122	4.8133	5.42	5.08	100.0
20000	6	Bubble	24.4411	3.7079	5.022	6.59	4.87	99.95
20000	7	Bubble	24.4411	3.4686	5.2578	7.05	4.65	99.97
20000	8	Bubble	24.4411	2.8473	5.6026	8.58	4.36	100.0

61. Резултати за податочен сет од 20 000 елементи

Со понатамошно зголемување на податочниот сет на 20 000 и 50 000 елементи, *Bubble Sort* се приближува до идеално забрзување, а во одредени случаи се појавува и т.н. *superlinear speedup*. Овој ефект може да се објасни со подобра

кеш-локалност и поефикасна распределба на податоците во меморијата при паралелно извршување. Сепак, поради неговата квадратична сложеност, сериското време достигнува 150-200 секунди, што го прави понатамошното тестирање практично неоправдано. Затоа алгоритмот е исклучен од понатамошните испитувања.

Dataset	Workers	Algorithm	Serial	Thread	Process	Thread Speedup	Process Speedup	Load Eff(%)
50000	2	Merge	0.2021	0.1743	2.4853	1.16	0.08	100.0
50000	3	Merge	0.2021	0.1815	2.6342	1.11	0.08	99.99
50000	4	Merge	0.2021	0.1896	3.2571	1.07	0.06	100.0
50000	5	Merge	0.2021	0.1901	3.5318	1.06	0.06	100.0
50000	6	Merge	0.2021	0.1783	3.8212	1.13	0.05	99.98
50000	7	Merge	0.2021	0.1917	4.2941	1.05	0.05	99.93
50000	8	Merge	0.2021	0.1898	4.85	1.06	0.04	100.0
50000	2	Quick	0.1442	0.1312	2.1945	1.1	0.07	100.0
50000	3	Quick	0.1442	0.1508	2.5617	0.96	0.06	99.99
50000	4	Quick	0.1442	0.1429	2.9566	1.01	0.05	100.0
50000	5	Quick	0.1442	0.154	3.4209	0.94	0.04	100.0
50000	6	Quick	0.1442	0.1469	3.8895	0.98	0.04	99.98
50000	7	Quick	0.1442	0.1685	4.3569	0.86	0.03	99.93
50000	8	Quick	0.1442	0.1611	4.8804	0.9	0.03	100.0
50000	2	Bubble	155.7589	94.162	49.2439	1.65	3.16	100.0
50000	3	Bubble	155.7589	48.2089	25.8524	3.23	6.02	99.99
50000	4	Bubble	155.7589	38.2471	20.8396	4.07	7.47	100.0
50000	5	Bubble	155.7589	29.1821	13.743	5.34	11.33	100.0
50000	6	Bubble	155.7589	24.9049	12.9371	6.25	12.04	99.98
50000	7	Bubble	155.7589	21.1102	11.1798	7.38	13.93	99.93
50000	8	Bubble	155.7589	19.9853	13.2148	7.79	11.79	100.0

62. Резултати за податочен сет од 50 000 елементи

Dataset	Workers	Algorithm	Serial	Thread	Process	Thread Speedup	Process Speedup	Load Eff(%)
100000	2	Merge	0.377	0.37	2.3396	1.02	0.16	100.0
100000	3	Merge	0.377	0.3443	2.5616	1.1	0.15	100.0
100000	4	Merge	0.377	0.3509	3.063	1.07	0.12	100.0
100000	5	Merge	0.377	0.3531	3.6429	1.07	0.1	100.0
100000	6	Merge	0.377	0.5647	5.5865	0.67	0.07	99.98
100000	7	Merge	0.377	0.3688	5.7254	1.02	0.07	99.97
100000	8	Merge	0.377	0.5679	6.9464	0.66	0.05	100.0
100000	2	Quick	0.5259	0.4359	2.6272	1.21	0.2	100.0
100000	3	Quick	0.5259	0.4055	4.1629	1.3	0.13	100.0
100000	4	Quick	0.5259	0.3847	3.7865	1.37	0.14	100.0
100000	5	Quick	0.5259	0.4046	4.2772	1.3	0.12	100.0
100000	6	Quick	0.5259	0.6184	4.6816	0.85	0.11	99.98
100000	7	Quick	0.5259	0.3454	5.0245	1.52	0.1	99.97
100000	8	Quick	0.5259	0.4447	5.6187	1.18	0.09	100.0

63. Резултати за податочен сет од 100 000 елементи

Во продолжение анализата се фокусира исклучиво на *Merge Sort* и *Quick Sort*. Кај сет од 100 000 елементи се уште не се забележува значајно забрзување во однос на сериското извршување, иако балансираноста на задачите е значително подобрена. Ова укажува дека самата рамномерна распределба не е доволен услов за ефикасна паралелизација доколку извршниот товар не е доволно голем.

Dataset	Workers	Algorithm	Serial	Thread	Process	Thread Speedup	Process Speedup	Load Eff(%)
500000	2	Merge	2.2048	2.1841	3.5686	1.01	0.62	100.0
500000	3	Merge	2.2048	2.1324	3.6608	1.03	0.6	100.0
500000	4	Merge	2.2048	2.199	3.9473	1.0	0.56	100.0
500000	5	Merge	2.2048	2.2877	5.1024	0.96	0.43	100.0
500000	6	Merge	2.2048	2.2222	5.6183	0.99	0.39	100.0
500000	7	Merge	2.2048	2.1472	5.9367	1.03	0.37	100.0
500000	8	Merge	2.2048	2.18	6.0622	1.01	0.36	100.0
500000	2	Quick	1.8425	1.6935	3.9755	1.09	0.46	100.0
500000	3	Quick	1.8425	2.2636	3.4597	0.81	0.53	100.0
500000	4	Quick	1.8425	1.7075	3.8003	1.08	0.48	100.0
500000	5	Quick	1.8425	1.7729	6.136	1.04	0.3	100.0
500000	6	Quick	1.8425	2.4275	6.501	0.76	0.28	100.0
500000	7	Quick	1.8425	2.401	6.754	0.77	0.27	100.0
500000	8	Quick	1.8425	2.4879	6.0148	0.74	0.31	100.0

64. Резултати за податочен сет од 500 000 елементи

Кај 500 000 елементи започнува минимално забрзување кај имплементацијата со нишки, додека процесите и понатаму не покажуваат значајна добивка. Во овој случај, балансираноста достигнува 100% за сите конфигурации, што значи дека ограничувањето не произлегува од распределбата, туку од архитектурни фактори. Со понатамошно зголемување на податоците се забележува стагнација кај threading имплементацијата, што директно се поврзува со Global Interpreter Lock (GIL) во Python. Бидејќи GIL не дозволува вистинска паралелност кај CPU-bound задачи, забрзувањето е суштински ограничено, без оглед на бројот на нишки.

Dataset	Workers	Algorithm	Serial	Thread	Process	Thread Speedup	Process Speedup	Load Eff(%)
1000000	2	Merge	4.6293	4.4785	4.7978	1.03	0.96	100.0
1000000	3	Merge	4.6293	4.5804	4.7616	1.01	0.97	100.0
1000000	4	Merge	4.6293	4.3194	4.9211	1.07	0.94	100.0
1000000	5	Merge	4.6293	4.6457	5.7835	1.0	0.8	100.0
1000000	6	Merge	4.6293	4.404	5.7683	1.05	0.8	100.0
1000000	7	Merge	4.6293	4.6322	6.4516	1.0	0.72	100.0
1000000	8	Merge	4.6293	4.2649	7.0391	1.09	0.66	100.0
1000000	2	Quick	3.5668	3.7268	4.6463	0.96	0.77	100.0
1000000	3	Quick	3.5668	3.5018	4.4995	1.02	0.79	100.0
1000000	4	Quick	3.5668	3.8256	4.9098	0.93	0.73	100.0
1000000	5	Quick	3.5668	3.6403	7.5624	0.98	0.47	100.0
1000000	6	Quick	3.5668	5.0056	7.2614	0.71	0.49	100.0
1000000	7	Quick	3.5668	4.8753	6.822	0.73	0.52	100.0
1000000	8	Quick	3.5668	4.4234	6.9446	0.81	0.51	100.0

65. Резултати за податочен сет од 1 000 000 елементи

Поради ова ограничување, во понатамошните експерименти се разгледува исклучиво multiprocessing пристапот. Кај сет од 3 000 000 елементи се забележува умерено забрзување од околу 1.5x, што укажува дека дури и кај големи податоци системските трошоци – комуникација, копирање на податоци и синхронизација – остануваат значаен фактор.

Dataset	Workers	Algorithm	Serial	Thread	Process	Thread Speedup	Process Speedup	Load Eff(%)
3000000	2	Merge	15.4482	-	11.6738	-	1.32	100.0
3000000	3	Merge	15.4482	-	10.1814	-	1.52	100.0
3000000	4	Merge	15.4482	-	10.0243	-	1.54	100.0
3000000	5	Merge	15.4482	-	10.687	-	1.45	100.0
3000000	6	Merge	15.4482	-	11.4316	-	1.35	100.0
3000000	7	Merge	15.4482	-	11.3132	-	1.37	100.0
3000000	8	Merge	15.4482	-	12.4336	-	1.24	100.0
3000000	2	Quick	13.5085	-	11.223	-	1.2	100.0
3000000	3	Quick	13.5085	-	10.998	-	1.23	100.0
3000000	4	Quick	13.5085	-	10.9264	-	1.24	100.0
3000000	5	Quick	13.5085	-	11.3575	-	1.19	100.0
3000000	6	Quick	13.5085	-	12.0263	-	1.12	100.0
3000000	7	Quick	13.5085	-	13.6722	-	0.99	100.0
3000000	8	Quick	13.5085	-	13.6911	-	0.99	100.0

66. Резултати за податочен сет од 3 000 000 елементи

Сличен тренд се забележува и кај 5 000 000 елементи, при што паралелизацијата носи ограничена, но стабилна добивка.

Dataset	Workers	Algorithm	Serial	Thread	Process	Thread Speedup	Process Speedup	Load Eff(%)
5000000	2	Merge	28.2338	-	22.0815	-	1.28	100.0
5000000	3	Merge	28.2338	-	21.1753	-	1.33	100.0
5000000	4	Merge	28.2338	-	15.9623	-	1.77	100.0
5000000	5	Merge	28.2338	-	15.8971	-	1.78	100.0
5000000	6	Merge	28.2338	-	17.1278	-	1.65	100.0
5000000	7	Merge	28.2338	-	16.7931	-	1.68	100.0
5000000	8	Merge	28.2338	-	16.7961	-	1.68	100.0
5000000	2	Quick	24.8575	-	18.6833	-	1.33	100.0
5000000	3	Quick	24.8575	-	17.6173	-	1.41	100.0
5000000	4	Quick	24.8575	-	18.1369	-	1.37	100.0
5000000	5	Quick	24.8575	-	23.6104	-	1.05	100.0
5000000	6	Quick	24.8575	-	20.8678	-	1.19	100.0
5000000	7	Quick	24.8575	-	19.8712	-	1.25	100.0
5000000	8	Quick	24.8575	-	19.6414	-	1.27	100.0

67. Резултати за податочен сет од 5 000 000 елементи

Генерално, резултатите покажуваат дека исплатливоста на паралелизацијата директно зависи од односот меѓу алгоритамската сложеност и системскиот overhead. Кај алгоритми со висока временска сложеност ефектот од паралелизацијата е поизразен, додека кај поефикасни алгоритми добивката е ограничена и силно зависна од архитектурните ограничувања на околината во која се извршуваат.

7.6. Интерпретација на резултатите

Автоматскиот режим на извршување, резултатите на крај ги мапира во *heatmap*. Кодот со кој е овозможена оваа функционалност со помош на Python библиотеката *pandas* изгледа вака:

```

def plot_heatmaps(self):
    if not self.results:
        return

    df = pd.DataFrame(self.results)

    for algo in df['algorithm'].unique():
        subset = df[df['algorithm'] == algo]

        # Thread heatmap
        try:
            pivot_thread = subset.pivot(index='dataset', columns='workers', values='thread_speedup')
            plt.figure(figsize=(10, 6))
            sns.heatmap(pivot_thread, annot=True, fmt=".2f", cmap="YlGnBu",
                        cbar_kws={'label': 'Speedup'})
            plt.title(f"Thread Speedup Heatmap - {algo}")
            plt.ylabel("Dataset size")
            plt.xlabel("Number of workers")
            plt.show()
        except:
            pass

        # Process heatmap
        try:
            pivot_process = subset.pivot(index='dataset', columns='workers', values='process_speedup')
            plt.figure(figsize=(10, 6))
            sns.heatmap(pivot_process, annot=True, fmt=".2f", cmap="YlOrRd",
                        cbar_kws={'label': 'Speedup'})
            plt.title(f"Process Speedup Heatmap - {algo}")
            plt.ylabel("Dataset size")
            plt.xlabel("Number of workers")
            plt.show()
        except:
            pass

```

68. Код за генерирање на топлотна мапа

Овој код овозможува напредна визуелна анализа на резултатите преку генерирање на *heatmap* графици за перформансите на алгоритмите. Неговата основна цел е да обезбеди јасна и интуитивна споредба на забрзувањата во зависност од големината на податочниот сет и бројот на работници.

На почетокот, резултатите од експериментите се трансформираат во табеларна структура *DataFrame*, со што се овозможува нивна системска обработка. Потоа, за секој алгоритам поединечно се издвојуваат релевантните податоци, што значи дека се анализира секој алгоритам независно од останатите.

Во рамките на секој алгоритам се креираат две посебни *heatmap* визуелизации. Првата ја претставува добивката во перформанси при извршување со нишки (thread speedup), а втората при извршување со процеси (process speedup).

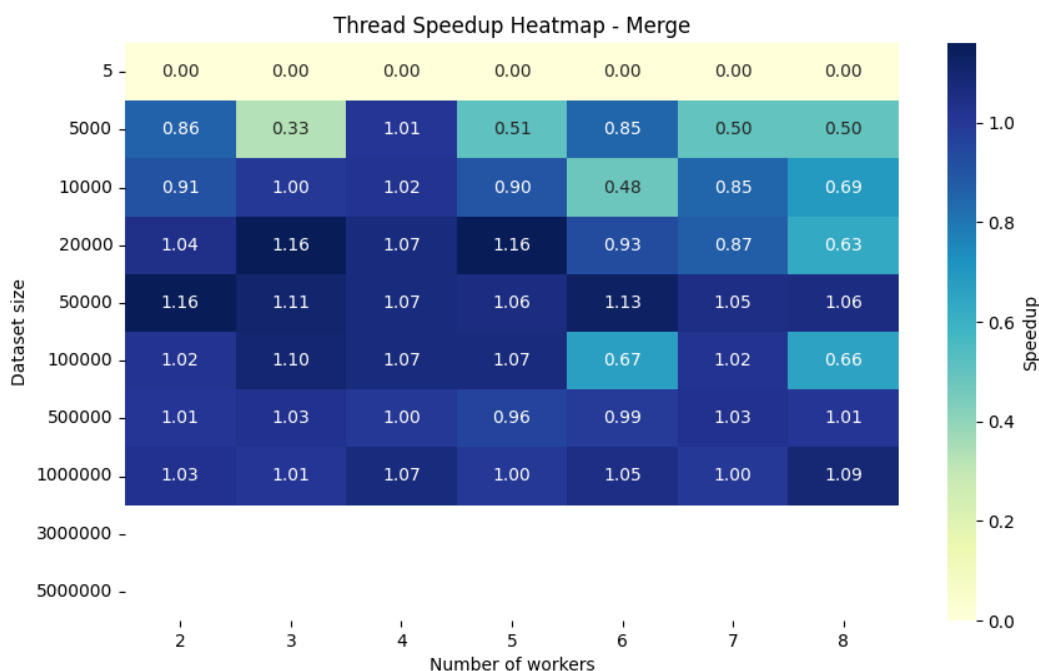
Податоците се реорганизираат во матрична форма така што редовите ги претставуваат големините на податочните сетови, колоните го претставуваат бројот на работници, а вредностите во матрицата ја претставуваат измерената speedup вредност.

Со помош на *heatmap* приказот, секоја комбинација од “големина на сет-број на работници” добива визуелна репрезентација преку боја, при што интензитетот на бојата директно корелира со постигнатата добивка во перформанси. Дополнително, вредностите се прикажани и нумерички во самите полиња, што овозможува истовремено квантитативна и визуелна анализа.

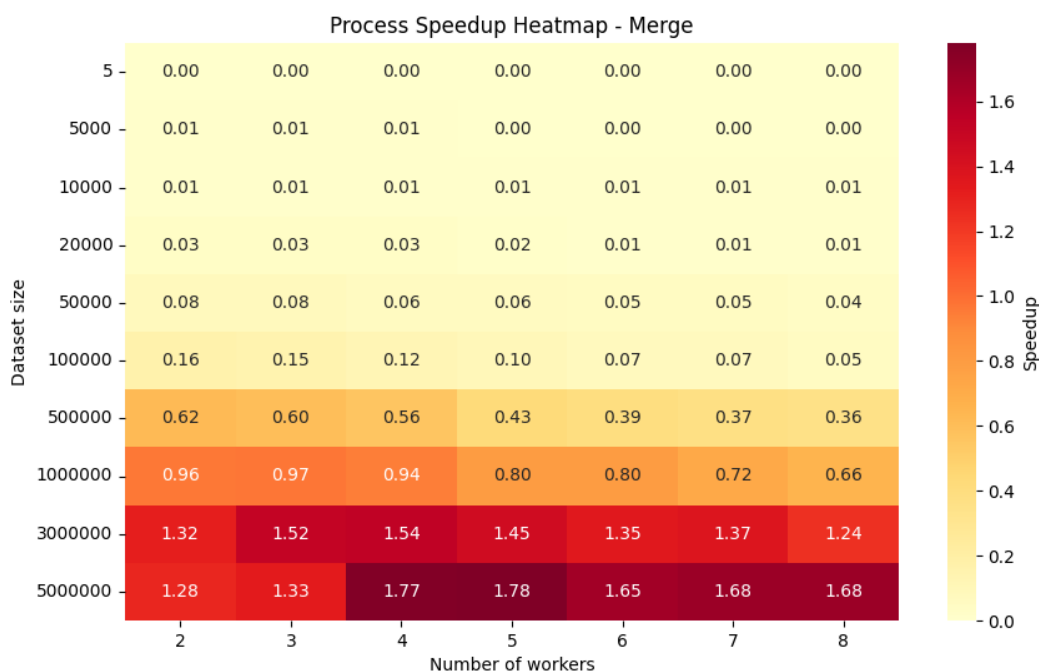
Ваквиот пристап значително ја олеснува идентификацијата на трендовите во податоците. На пример, може лесно да се воочи при кои големини на податочен сет паралелизацијата носи најголема корист, дали зголемувањето на бројот на работници води до пропорционален раст на вредноста на забрзувањето, како и во кои случаи доаѓа до заситување или намалување на ефикасноста.

Со други зборови, овој код овозможува комплетна визуелна мапа на перформансите на алгоритмите во автоматскиот режим, претставувајќи ја најкондензираната и најпрегледна форма на резултатите од целокупната експериментална анализа.

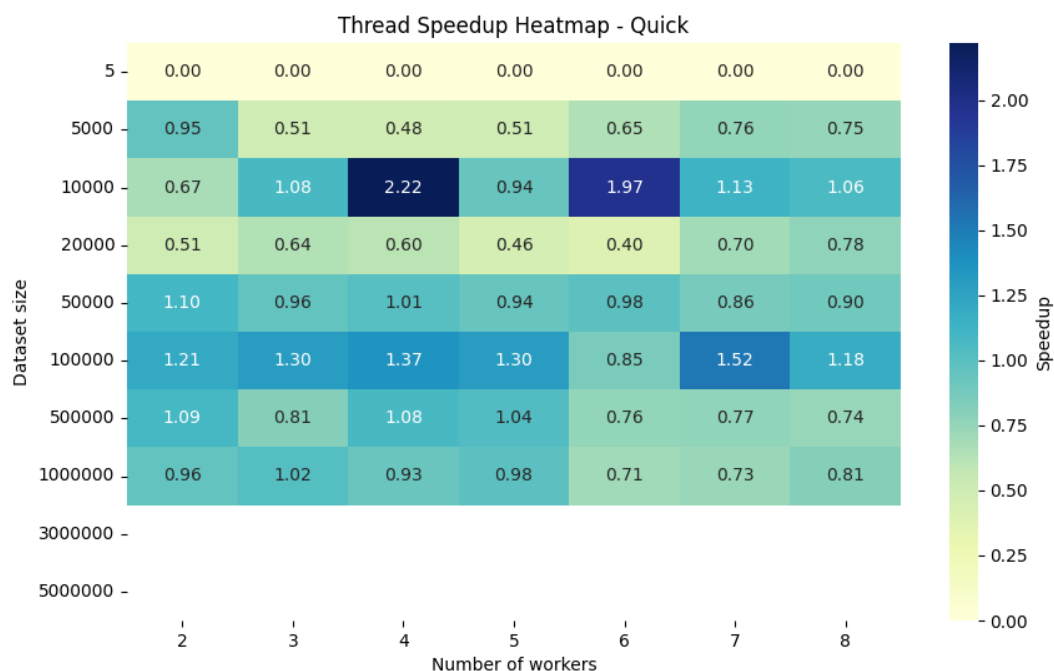
Во продолжение следува аналитичка интерпретација на прикажаните топлотни мапи, со фокус на трендовите, скалабилноста и архитектурните ограничувања.



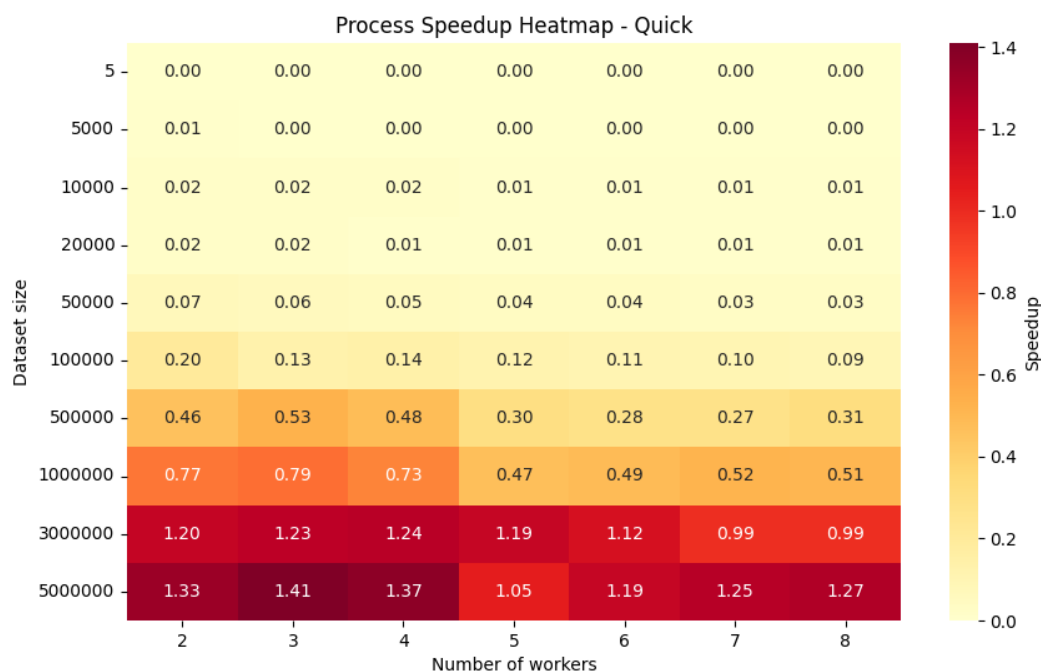
Анализата на топлотната мапа за **Thread Speedup – Merge Sort** покажува дека кај малите податочни сетови (5, 5 000, 10 000 елементи) практично нема реално забрзување. Вредностите се блиску до 1 или под 1, што значи дека паралелното извршување со нишки не носи добивка во однос на сериското. Кај средните сетови (20 000, 50 000) се забележува благо подобрување, со вредности околу 1.10-1.16, што укажува на минимална ефикасност. Меѓутоа, со понатамошно зголемување на податоците (500 000, 1 000 000), speedup вредностите повторно се стабилизираат околу 1.00-1.07. Овој образец јасно укажува на ограничувањето предизвикано од GIL, кој спречува вистинска паралелност кај CPU-bound операциите. Зголемувањето на бројот на нишки не доведува до пропорционално зголемување на перформансите, што значи дека стабилноста е суштински ограничена.



Топлотната мапа за **Process Speedup – Merge Sort** покажува поинаков тренд. Кај малите и средните сетови (до 100 000 елементи) вредностите се значително под 1, што значи дека multiprocessing извршувањето е побавно од сериското. Ова директно го потврдува влијанието на overhead-от од креирање процеси и меѓупроцесна комуникација. Меѓутоа, кај големите сетови (3 000 000 – 5 000 000) се покажува јасен позитивен тренд. Забрзувањето достигнува вредности од околу 1.5x кај 3 милиони елементи и до приближно 1.7-1.8 кај 5 милиони елементи. Ова покажува дека дури при доволно голем извршен товар multiprocessing пристапот успева да го амортизира комуникацискиот трошок и да обезбеди реална добивка. Сепак, скалабилноста не е линеарна – зголемувањето на бројот на процеси над одредена точка не носи пропорционално подобрување, што укажува на мемориски и синхронизациски ограничувања.



Kaj **Thread Speedup – Quick Sort** се забележува нешто поразлична динамика. Kaj некои средни податочни сетови (на пример 10 000 и 100 000 елементи) се појавуваат повисоки вредности на speedup, дури и над 2x во одредени конфигурации. Овие резултати можат да бидат последица на поволна распределба на податоците и кеш-ефекти, но не претставуваат стабилен тренд. Kaj поголемите сетови вредностите повторно се движат околу 1, а во одредени случаи и под 1. Тоа повторно ја потврдува тезата дека threading kaj Python не овозможува вистинска паралелност за CPU-bound алгоритми и дека забележаните подобрувања се ограничени и нестабилни.

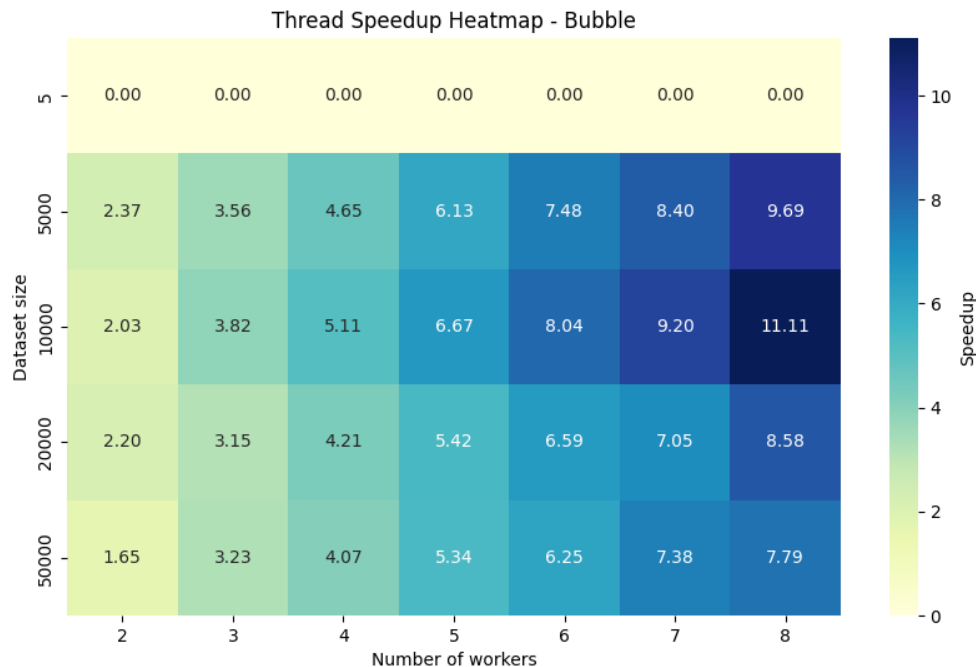


Кај **Process Speedup – Quick Sort**, топлотната мапа јасно покажува дека за сите податочни сетови до 1 000 000 елементи speedup вредностите се под 1, без оглед на бројот на процеси. Ова значи дека паралелното извршување е побавно од сериското. Причината е што кај алгоритам со сложеност $O(n \log n)$, за овие големини на податоци времето на сериско извршување е релативно кратко, а трошокот за креирање процеси, копирање на податоци и меѓупроцесна комуникација не може да се амортизира.

Кај сет од 3 000 000 елементи се појавува првото реално забрзување, при што максималната вредност е 1.24 за 4 процеси. Ова укажува дека токму при оваа конфигурација постои оптимален баланс меѓу извршниот товар и комуникацискиот overhead. Зголемувањето над 4 процеси не носи дополнително подобрување, што укажува на појава на заситување и зголемени синхронизациски трошоци.

Кај сет од 5 000 000 елементи максималното забрзување изнесува 1.41 и се постигнува со 2 процеси. Интересно е што најдобриот резултат не е кај најголем број процеси, што дополнително потврдува дека Quick Sort во оваа имплементација не скалира линеарно. Поголем број процеси воведува

дополнителна координација и поделба на подзадачи, што ја намалува ефикасноста. Овој образец укажува дека постои оптимална точка на паралелизација, по кој маргиналната добивка опаѓа.



Кај **Thread Speedup – Bubble Sort** се забележува јасен и постепен тренд на зголемување на забрзувањето со растот на бројот на нишки и со растот на податочниот сет.

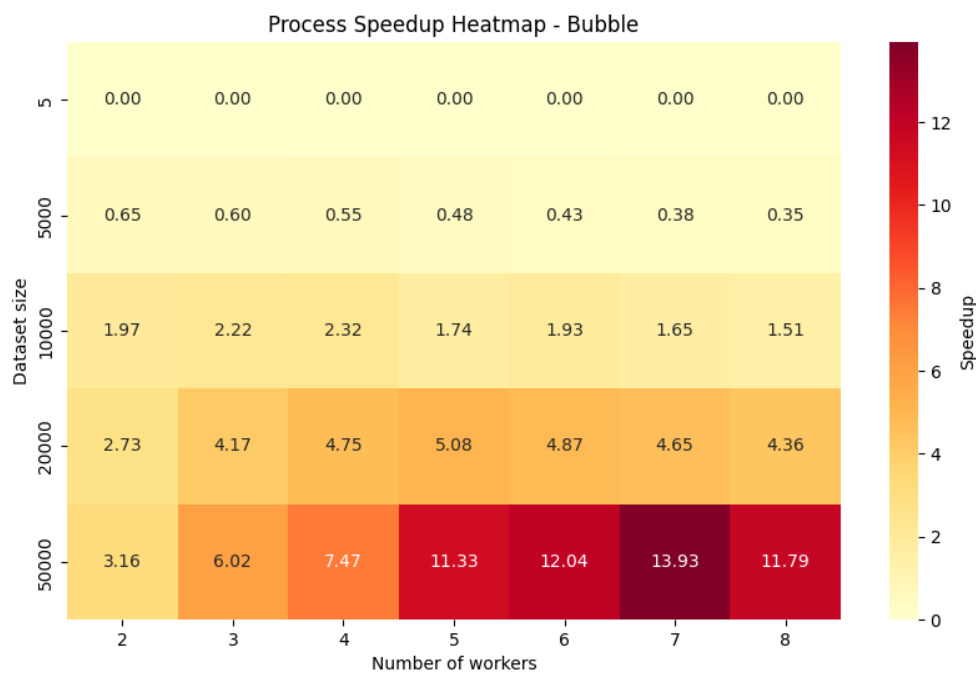
За сет од 5 000 елементи вредностите растат од 2.37 (2 нишки) до 9.69 (8 нишки). Ова претставува значајно и речиси линеарно забрзување, што покажува дека кај алгоритам со квадратична сложеност постои доволно извршен товар за да се почувствува ефектот од поделба на задачите.

Кај 10 000 елементи трендот е уште поизразен, со максимум од 11.11 кај 8 нишки. Ова укажува дека со зголемување на податоците overhead –от станува релативно занемарлив во однос на вкупното време на извршување.

Кај 20 000 елементи вредностите и понатаму растат стабилно, достигнувајќи 8.58 кај 8 нишки. Кај 50 000 елементи се забележува малку поумерен раст, со

максимум 7.79. ова може да укажува на почеток на заситување, односно ограничување од страна на мемориската пропусност или распоредувачот на оперативниот систем.

Генерално, thread speedup кај Bubble Sort покажува дека кај алгоритми со висока временска сложеност паралелизацијата може да донесе драматично подобрување, особено кога извршниот товар е доволно голем за да го оправда overhead –от.



Кај **Process Speedup Bubble Sort** се забележува уште поизразена динамика. За сет од 10 000 елементи вредностите се движат од 1.97 до максимум 2.32 (кај 4 процеси), по што следи опаѓање. Ова покажува дека постои оптимален број процеси, по кој дополнителните процеси внесуваат повеќе трошок отколку добивка.

Кај 20 000 елементи се забележува значително подобрување, со максимум 5.08 кај 5 процеси. Овој резултат укажува на значително подобрена амортизација на overhead –от.

Најизразен ефект се гледа кај 50 000 елементи, каде speedup вредностите достигнуваат 13.93 кај 7 процеси. Ова претставува *superlinear speedup* и може да се објасни со подобрена кеш-локалност, помали работни подмножества во секој процес и поефикасно користење на мемориската хиерархија. Сепак, кај 8 процеси вредноста опаѓа на 11.79, што укажува на заситување и зголемена координациска цена.

Анализата на овие топлотни мапи потврдува неколку клучни принципи:

- Алгоритмите со висока временска сложеност ($O(n^2)$) покажуваат значајна корист од паралелизацијата;
- Алгоритмите со $O(n \log n)$ сложеност имаат ограничена скалабилност освен кај многу големи податочни сетови;
- Постои оптимален број на процеси/нишки за секоја големина на податочен сет;
- По одреден точка се јавува заситување и маргиналната добивка опаѓа;
- Multiprocessing овозможува реална паралелност, но по цена на комуникациски overhead;
- Threading кај CPU-bound задачи може да даде добри резултати кај специфични сценарија, но не обезбедува стабилна и линеарна скалабилност.

Овој дел од анализата ја заокружува сликата дека ефикасноста на паралелните алгоритми е резултат на комплексна интеракција меѓу алгоритамската сложеност, архитектурата на извршната околина и системските трошоци на паралелизација.

8. Заклучок

Python нуди богата инфраструктура за конкурентно програмирање, но изборот меѓу нишки и процеси зависи од природата на задачата. Нишките се идеални за задачи кои вклучуваат чекање на надворешни ресурси, додека процесите се задолжителни за задачи кои барат масивни пресметки. Модулот

concurrent.futures ги унифицира и поедноставува двата пристапи, што го прави паралелното програмирање подостапно и поконтролирано.

Со правилен избор на моделот, програмерот може да постигне значителни подобрувања во перформансите, скалабилноста и ефикасноста на Python апликациите.

9. Користена литература

- Python Software Foundation. (2025). *Python Documentation*. Python.org – Python: Монографија. Достапно на: <https://docs.python.org/3/>
- Python Software Foundation. (2025). *multiprocessing — Process-based parallelism*. Python.org – Python: Монографија. Достапно на: <https://docs.python.org/3/library/multiprocessing.html>
- Python Software Foundation. (2025). *concurrent.futures — Launching parallel tasks*. Python.org – Python: Монографија. Достапно на: <https://docs.python.org/3/library/concurrent.futures.html>
- Паралелно програмирање – скрипта со примери, Милош Иванович, Крагујевац 2016